Martin Bergstø

# Ontology-Driven Decision Support for Machine Learning Method Selection in Product Development and Production

## Combining Semantic Literature Retrieval with Ontology-Based Filtering and Implementation Support

Master's thesis in Engineering and ICT
Supervisor: Andrei Lobov
June 2026

Norwegian University of Science and Technology
Faculty of Engineering
Department of Mechanical and Industrial Engineering

NTNU

# ABSTRACT

Write an abstract/summary of your thesis, and state your main findings here.

A summary should be included in both English and any second language, if this is applicable, regardless if the thesis is written in English or in your preferred language. These should be on separate pages, the English version first.

# PREFACE

Write the preface of your thesis here.

You may include acknowledgements and thanks as part of your preface on this page, or you may add it as a new chapter after the preface.

# CONTENTS

**Abstract** i

**Preface** ii

**Contents** v

**List of Figures** v

**List of Tables** vii

**Abbreviations** ix

**1 Introduction** 1

1.1 Motivation . . . . . 1

1.2 Problem Description . . . . . 2

1.3 Research Questions . . . . . 3

1.4 Thesis Structure . . . . . 4

**2 Background and Related Work** 5

2.1 Machine Learning . . . . . 5

2.1.1 Machine Learning Paradigms and Tasks . . . . . 5

2.1.2 Machine Learning Methods . . . . . 7

2.1.3 Neural Networks and Deep Learning . . . . . 11

2.1.4 Attributes Relevant for Method Selection (MISSING) . . . . 13

2.1.5 Approaches to Method Selection . . . . . 14

2.2 Machine Learning in Product Development and Production . . . . . 15

2.2.1 Applications of Machine Learning . . . . . 15

2.2.2 Learning Paradigms in Industrial Applications . . . . . 16

2.2.3 Pre-Master Findings . . . . . 16

2.3 Ontologies and Knowledge Representation . . . . . 17

2.3.1 Ontology Description . . . . . 17
2.3.2 Why ML and Ontology . . . . . 18
2.3.3 Semantic Web Standards . . . . . 18
2.3.4 Existing Machine Learning Ontologies . . . . . 19
2.3.5 ML Ontology Structure and Content . . . . . 22
2.4 Semantic Retrieval and Text Similarity . . . . . 26
2.4.1 Text Embeddings and Semantic Similarity . . . . . 26
2.4.2 Transformer-Based Sentence Embedding Models . . . . . 27
2.4.3 Semantic Retrieval . . . . . 28
2.4.4 Cross-Encoder Reranking . . . . . 29
2.4.5 Scientific Literature as a Knowledge Source . . . . . 31
2.5 User Feedback in Recommendation Systems . . . . . 31
2.6 Related Work and Existing Tools . . . . . 32
2.6.1 Ontology-Based Meta AutoML . . . . . 32
2.6.2 Template-Based Code Generation . . . . . 33
2.6.3 AI-on-Demand Platform . . . . . 33

**3 Methodology and System Overview 35**
3.1 Design Science Research . . . . . 35
3.2 System Overview . . . . . 37
3.3 Design Goals and Motivation . . . . . 37
3.4 Incremental and Iterative Development . . . . . 38
3.5 User Testing . . . . . 39
3.5.1 Setup and Context . . . . . 39

**4 Implementation 43**
4.1 System Architecture . . . . . 43
4.2 Knowledge Graph and Database . . . . . 46
4.2.1 ML Ontology Extension . . . . . 46
4.2.2 Database Structure . . . . . 52
4.3 Recommendation Process . . . . . 53
4.3.1 Ontology-Based Candidate Retrieval . . . . . 55
4.3.2 Article Similarity Scoring . . . . . 56
4.3.3 Method Scoring . . . . . 57
4.3.4 Feedback Boost . . . . . 57
4.4 Notebook Generation . . . . . 59
4.5 Article Retrieval and Knowledge Extraction . . . . . 59
4.6 Web Interface . . . . . 59

**5 Results 61**

5.1 Project 1 (P1): Unsupervised Learning and Anomaly Detection for Time Series Data . . . . . 61
5.2 Project 2 (P2): Reinforcement Learning Across Product Lifecycle . 64

**6 Discussion** **67**
6.0.1 Feedback and Limitations . . . . . 67

**7 Conclusions** **69**

**References** **71**

**Appendices:** **71**

# LIST OF FIGURES

2.1.1 ML paradigms . . . . . 6
2.1.2 ML methods by paradigm and task . . . . . 10
2.1.3 Neural network architectures . . . . . 13
2.3.1 ML ontologies comparison . . . . . 21
2.3.2 ML Ontology schema . . . . . 22
2.3.3 ML Ontology classes . . . . . 23
2.3.4 ML task hierarchy . . . . . 25
2.4.1 Embedding space example . . . . . 26
2.4.2 SBERT architecture . . . . . 28
2.4.3 Keyword vs. dense retrieval . . . . . 29
2.4.4 Bi-encoder vs. cross-encoder . . . . . 30

3.1.1 DSR methodology . . . . . 36
3.2.1 MLguide system overview . . . . . 37
3.3.1 ML selection roadmap . . . . . 38
3.4.1 Incremental development process . . . . . 39
3.5.1 Development timeline . . . . . 41

4.1.1 MLguide component diagram . . . . . 44
4.1.2 MLguide layered architecture . . . . . 45
4.1.3 MLguide package diagram . . . . . 46
4.2.1 ML Ontology UML diagram . . . . . 48
4.2.2 Article instance example . . . . . 49
4.2.3 PostgreSQL ER diagram . . . . . 53
4.3.1 Recommendation process sequence diagram . . . . . 54
4.3.2 Ranking flow . . . . . 58
4.4.1 Notebook generation sequence . . . . . 59
4.5.1 Article retrieval flow . . . . . 59

5.1.1 P1 group participation . . . . . 61
5.1.2 P1 timing of MLguide use . . . . . 62
5.1.3 P1 recommendation overlap . . . . . 62
5.1.4 P1 timing and overlap . . . . . 63
5.1.5 P1 recommended vs. used . . . . . 63
5.1.6 P1 reported benefits . . . . . 63
5.1.7 P1 reported issues . . . . . 64
5.2.1 P2 group participation . . . . . 64
5.2.2 P2 how groups used MLguide . . . . . 65
5.2.3 P2 key benefits . . . . . 65
5.2.4 P2 methods vs. recommended . . . . . 65

# LIST OF TABLES

2.3.1 ML_approach properties . . . . . 24
2.3.2 ML_approach property coverage . . . . . 24

3.5.1 User testing sessions . . . . . 40

4.2.1 New ontology classes . . . . . 47
4.2.2 New ML_approach instances . . . . . 50
4.2.3 ML_approach property coverage after extension . . . . . 51
4.2.4 ML_task_family instances . . . . . 52
4.3.1 Ontology properties used in candidate retrieval . . . . . 55

# ABBREVIATIONS

List of all abbreviations in alphabetic order:

- **AIoDP** AI-on-Demand Platform
- **AutoML** Automated Machine Learning
- **BERT** Bidirectional Encoder Representations from Transformers
- **BM25** Best Match 25
- **HPC** High-Performance Computing
- **IR** Information Retrieval
- **ML** Machine Learning
- **NTNU** Norwegian University of Science and Technology
- **OMA-ML** Ontology-Based Meta AutoML
- **OWL** Web Ontology Language
- **OWL-DL** OWL Description Logic
- **PROV-O** Provenance Ontology
- **RDF** Resource Description Framework
- **RDFS** RDF Schema
- **SBERT** Sentence-BERT
- **SHACL** Shapes Constraint Language
- **SKOS** Simple Knowledge Organization System
- **SME** Small and Medium-sized Enterprise
- **SPARQL** SPARQL Protocol and RDF Query Language

- **SQL** Structured Query Language
- **tf-idf** Term Frequency-Inverse Document Frequency
- **W3C** World Wide Web Consortium

CHAPTER

# ONE

# INTRODUCTION

This chapter introduces the purpose and overall direction of the thesis. It presents the motivation for the study and describes the main problem that is addressed. The research questions are then defined to guide the work. Finally, the chapter provides an overview of the structure of the thesis.

## 1.1 Motivation

Machine learning has seen strong growth in recent years and is now widely applied in product development and production. It is used for tasks such as optimization, control, prediction, and decision support (**wuest2016ml_manufacturing**).

Advances in data availability and computing power have further increased the use of machine learning in industrial settings (**rai2021ml_manufacturing_industry4**). As a result, machine learning plays an important role in improving efficiency, supporting decision-making, and enabling new capabilities in engineering and manufacturing.

At the same time, the field of machine learning is large and diverse. Many different algorithms, methods, and approaches are available, each with its own strengths and weaknesses (**wuest2016ml_manufacturing**). There is no single method that performs best for all problems, and performance depends on the specific application and data (**adekoya2022applications_ai_em**; **lee2020machine_learning_enterpri**

This creates a challenge for engineers and practitioners, who must select suitable methods for their problems. The large number of available options can act as a barrier and limit the effective use of machine learning in practice (**adekoya2022applications_ai_e**

Selecting an appropriate method often requires balancing multiple factors such as accuracy, interpretability, and data requirements (**plathottam2023review_ai_manufacturing**; **ali2017accurate_multi_criteria_decision_making_methodology_recommending_machi**

If the selection is not done carefully, the resulting model may perform poorly or fail to provide useful insights (**ali2017accurate_multi_criteria_decision_making_methodology_**
In practice, simpler models are often preferred due to their interpretability, even if more complex models may offer higher accuracy (**paleyes2022challenges_deploying_ml_survey_**
This highlights the importance of considering trade-offs in real-world applications.

Another challenge is the gap between research and practical implementation. Research results are distributed across many articles and domains, which makes it difficult to gain a complete overview (**adekoya2022applications_ai_em**). In addition, most studies focus on developing models, while less attention is given to how they should be selected and applied in practice.

Companies also face challenges related to lack of knowledge and experience when adopting machine learning (**sonntag2023pattern_ml_product_development**). In many cases, the selection of methods is based on trial and error, which is time-consuming and inefficient (**sonntag2023pattern_ml_product_development**). There is also often pressure to adopt machine learning without a clear plan, which increases the risk of unsuccessful implementations (**plathottam2023review_ai_manufacturing**).

These challenges highlight the need for more structured approaches that can support method selection and make machine learning more accessible.

## 1.2 Problem Description

Despite the growing adoption of machine learning in manufacturing and engineering, practitioners often lack adequate support when selecting appropriate methods for their specific problems. Although a large number of use cases exist, there is currently no guidance or standard for developing machine learning solutions from ideation to deployment, and it remains difficult for non-experts to begin developing solutions for their specific problems (**chen2023ml_manufacturing_industry4**). In practice, method selection is often based on trial and error, which is both time-consuming and dependent on prior experience that many practitioners do not have (**sonntag2023pattern_ml_product_development**).

A central limitation is that no structured approach currently exists for making an informed selection of machine learning algorithms based on a well-defined problem description (**sonntag2023pattern_ml_product_development**). Existing frameworks tend to use accuracy or prediction speed as the primary selection

criteria, which are often too general and can lead to unsuitable methods being chosen (**sala2018selecting_ml_algorithm**). Manual evaluation of candidate algorithms is highly time-consuming, and if the selection is not done carefully, the resulting model may fail to produce useful results (**ali2017accurate_multi_criteria_decision_**

Existing research further focuses primarily on the development and evaluation of machine learning models, while less attention is given to what conditions lead to selecting one approach over another in specific industrial contexts (**arena2024conceptual_frame**
The lack of interpretability in many approaches is also a key barrier to adoption, as practitioners need to understand and trust the recommendations they receive (**presciuttini2024ml_iot_manufacturing_operations**).

At the same time, the relevant knowledge is distributed across a large and fragmented body of literature. The extensive number of works on machine learning algorithms and applications creates broad knowledge on the topic, but may also generate disorientation when selecting the right approach (**sala2018selecting_ml_algorithm**).
This knowledge is difficult to access and apply systematically during method selection, which limits the ability to make informed, evidence-based decisions (**adekoya2022applications_ai_em**).

Addressing these challenges requires a structured approach that can translate problem descriptions into method recommendations and connect those recommendations to relevant research. This thesis investigates whether combining ontology-based knowledge representation with semantic retrieval from scientific literature can provide such support, offering interpretable recommendations grounded in prior research alongside guidance for early-stage implementation.

**Burde MLguide forklares i så stor detalj? Og bør det forklares her eller under research questions?**

## 1.3 Research Questions

The challenges outlined above point to a need for structured decision support that can connect problem descriptions with suitable machine learning methods, use the research literature to justify those recommendations, and lower the barrier to early-stage implementation. This thesis investigates whether a system combining these capabilities can provide useful and justified recommendations in practice. The following hypothesis is proposed:

> A decision-support system that combines ontology-based knowledge representation, semantic similarity between problem descriptions and scientific literature, and structured implementation support can pro-

vide relevant and justified machine learning method recommendations for users with limited machine learning expertise.

To investigate this hypothesis, the following research questions are defined:

- How can ontology-based knowledge improve machine learning method selection?
- How can semantic similarity between problem descriptions and literature support recommendations?
- How can literature-based evidence support and justify machine learning method recommendations?
- To what extent can such a system support early-stage machine learning implementation?

To investigate the hypothesis and research questions, a web-based decision-support system, called MLguide, was developed that combines structured knowledge with information from scientific literature.

## 1.4 Thesis Structure

The remainder of this thesis is structured as follows:

- Chapter 1 introduces the motivation, problem description, and research questions
- Chapter 2 presents background and related work
- Chapter 3 presents the methodological framework and system overview
- Chapter 4 describes the system development
- Chapter 5 describes the research approach
- Chapter 6 presents the system implementation
- Chapter 7 presents the results
- Chapter 8 discusses the findings and limitations
- Chapter 9 concludes the thesis and suggests future work

CHAPTER

# TWO

# BACKGROUND AND RELATED WORK

This chapter introduces the theoretical and technical background for the work presented in this thesis. It covers machine learning methods and their application in product development and production, ontologies and knowledge representation, and semantic retrieval from scientific literature. The chapter ends with a review of related tools and prior work.

## 2.1 Machine Learning

### 2.1.1 Machine Learning Paradigms and Tasks

Machine learning refers to algorithms that improve their performance at some task through experience (**mitchell1997machine_learning**). Such algorithms can be broadly categorized according to the type of supervision they receive during that experience, also called training. There are three main paradigms commonly distinguished in the literature: supervised learning, unsupervised learning, and reinforcement learning (**bishop2006pattern_recognition_machine_learning**) (**Goodfellow-e**
(**sarker2021machine_learning_algorithms_real_world_applications_research_dir**
The following sections outline each paradigm and introduce some common tasks.

In supervised learning, the training data consists of input vectors paired with corresponding target values. The two most common supervised tasks are classification, where the goal is to assign an input to one of a finite number of discrete categories, and regression, where the goal is to predict a continuous numerical value (**bishop2006pattern_recognition_machine_learning**) (**Goodfellow-et-al-2016**).

In unsupervised learning, no target values are provided. The algorithm must instead identify structure in the data directly (**bishop2006pattern_recognition_machine_learning**). Common tasks include clustering, dimensionality reduction, and anomaly detection (**sarker2021machine_learning_algorithms_real_world_applications_research_direc**) (**Goodfellow-et-al-2016**). Clustering groups data points such that objects within the same group are more similar to each other than to those in other groups (**sarker2021machine_learning_algorithms_real_world_applications_research_direction**). Dimensionality reduction simplifies data by reducing the number of features, which leads to better human interpretations and lower computational costs (**sarker2021machine_learning_**). Anomaly detection identifies observations that deviate significantly from the expected distribution, and is used in applications such as fraud detection and fault monitoring (**Goodfellow-et-al-2016**). Semi-supervised learning occupies a middle ground, operating on both labeled and unlabeled data, which is useful in contexts where labeled examples are scarce (**sarker2021machine_learning_algorithms_real_world_**). It is worth noting that the boundary between supervised and unsupervised learning is not always well-defined, and many methods can be applied across both types of problems (**Goodfellow-et-al-2016**).

Reinforcement learning differs fundamentally from the other paradigms. Rather than learning from a fixed dataset, an agent interacts with an environment and learns through trial and error, receiving rewards or penalties based on its actions (**bishop2006pattern_recognition_machine_learning**) (**sarker2021machine_learning_algor**). Within reinforcement learning, methods can be broadly divided according to what they learn. In value-based methods, a value function is learned from which the policy is derived either explicitly or implicitly, whereas in policy-based methods the policy is learned directly without the need for a value function (**sewak2019policy_based_reinforcem**) (**sutton2018reinforcement_learning**). A third class, actor-critic methods, combines both approaches by maintaining a separate policy structure and value function simultaneously (**sutton2018reinforcement_learning**).

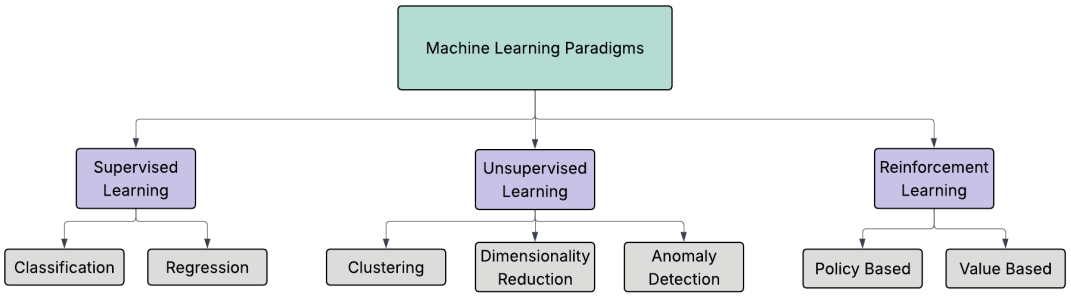


**Figure 2.1.1:** Overview of machine learning paradigms with common tasks.

### 2.1.2 Machine Learning Methods

The following sections introduce a selection of common and most established machine learning methods and place them within the paradigms and tasks described above. The aim is not to provide a comprehensive description of each method or to cover underlying mathematical details, but to give an overview of common method types and what fundamentally distinguishes them. The section then gives a brief overview of attributes that are relevant to consider when selecting a method for a given problem.

#### Supervised methods

Tree-based methods learn by recursively partitioning the input space according to feature values. A decision tree splits data into regions by asking a sequence of questions about the input features, assigning each region a predicted output. They can be used for both classification and regression tasks and require little data preparation, such as feature scaling (**geron2019hands_on_machine_learning**; **sarker2021machin**
Their decisions are intuitive and easy to interpret, as the classification rules they produce can even be applied manually (**geron2019hands_on_machine_learning**).
Random forests extend this by training many trees on different subsets of the data and aggregating their predictions, which makes them more robust and generally improves predictive accuracy (**breiman2001random_forests**). However, combining many trees makes the model harder to interpret than a single decision tree (**geron2019hands_on_machine_learning**). Gradient boosting methods such as XGBoost build trees sequentially to improve on the predictions of previous trees, and are known for strong generalisation performance on large datasets (**chen2016xgboost**; **sarker2021machine_learning_algorithms_real_world_applicati**

Linear and polynomial regression predict a continuous output and are among the most widely used methods for regression tasks (**sarker2021machine_learning_algorithms_**
Regularised variants constrain the model weights to improve generalisation. Ridge regression keeps weights small but never eliminates them entirely, while LASSO regression tends to set some weights exactly to zero, which effectively performs feature selection and produces a sparse model (**tibshirani1996regression_lasso_ridge**; **geron2019hands_on_machine_learning**). Logistic regression estimates the probability of class membership using a linear combination of input features, and is commonly used for classification tasks, particularly when a simple and interpretable model is preferred (**geron2019hands_on_machine_learning**; **sarker2021machine_learning_algorithms_real_world_applications_research_dire**

Support vector machines find a decision boundary that maximises the margin

between classes (**cortes1995support_vector_networks**). By applying kernel functions, SVMs can model non-linear decision boundaries and are effective in high-dimensional spaces. They can be used for both classification and regression tasks (**geron2019hands_on_machine_learning**; **sarker2021machine_learning_algorithms_**

K-nearest neighbours classifies a new data point based on the majority class among its closest neighbours in the training data, using a distance measure such as Euclidean distance. The method requires no explicit training step but can be computationally expensive at prediction time for large datasets (**cunningham2021k_nearest_neighbour_** **sarker2021machine_learning_algorithms_real_world_applications_research_directions**

### Unsupervised methods

Clustering algorithms group data points based on similarity without the use of labels. K-means partitions data into a fixed number of clusters by assigning each instance to the nearest centroid and updating the centroids iteratively until convergence (**geron2019hands_on_machine_learning**). The number of clusters must be specified in advance, and the algorithm is sensitive to the initial centroid placement (**geron2019hands_on_machine_learning**). DBSCAN takes a different approach by defining clusters as dense regions in the data space, which allows it to find clusters of arbitrary shape and identify outliers as noise without requiring the number of clusters to be specified in advance (**ester1996dbscan**; **sarker2021machine_learning_algorithms_real_world_applications_research_directions**

For dimensionality reduction, principal component analysis (PCA) identifies the axes of greatest variance in the data and projects it onto a lower-dimensional space defined by these axes, called principal components (**hotelling1932pca**; **geron2019hands_on_machine_learning**). This reduces computational cost and can aid visualisation while preserving as much of the original variance as possible (**geron2019hands_on_machine_learning**; **sarker2021machine_learning_algorithms_re**

Anomaly detection methods identify observations that deviate significantly from the expected distribution. Isolation Forest isolates anomalies by randomly partitioning the data, exploiting the fact that anomalies are few and structurally different from normal observations (**liu2008isolation_forest**). One-class SVM, a variant of the support vector machine, learns a boundary around normal data and flags observations outside this boundary as anomalies (**geron2019hands_on_machine_learning**).

### Reinforcement learning methods

Within reinforcement learning, methods differ primarily in what they learn. Value-based methods such as Q-learning estimate the expected return for each action in a

given state and derive a policy from these estimates (**watkins1992q_learning**). Policy-based methods such as policy gradient algorithms optimise the policy directly, which makes them better suited for problems with continuous or high-dimensional action spaces (**sutton1999policy_gradient_methods_reinforcement_learni**). Actor-critic methods combine both approaches by maintaining a separate policy and value function simultaneously (**zhang2020taxonomy_reinforcement_learning_algori**).

Figure 2.1.2 (on the following page) provides a structured overview of common machine learning methods organised by paradigm and task. It includes the methods described in the sections above, as well as additional commonly used methods not covered in detail here. The overview is based on a combination of taxonomies and categorisations described in the literature, and specifically draws on the works of **sarker2021machine_learning_algorithms_real_world_applications_research_dire**, **geron2019hands_on_machine_learning**, and **zhang2020taxonomy_reinforcement_l**

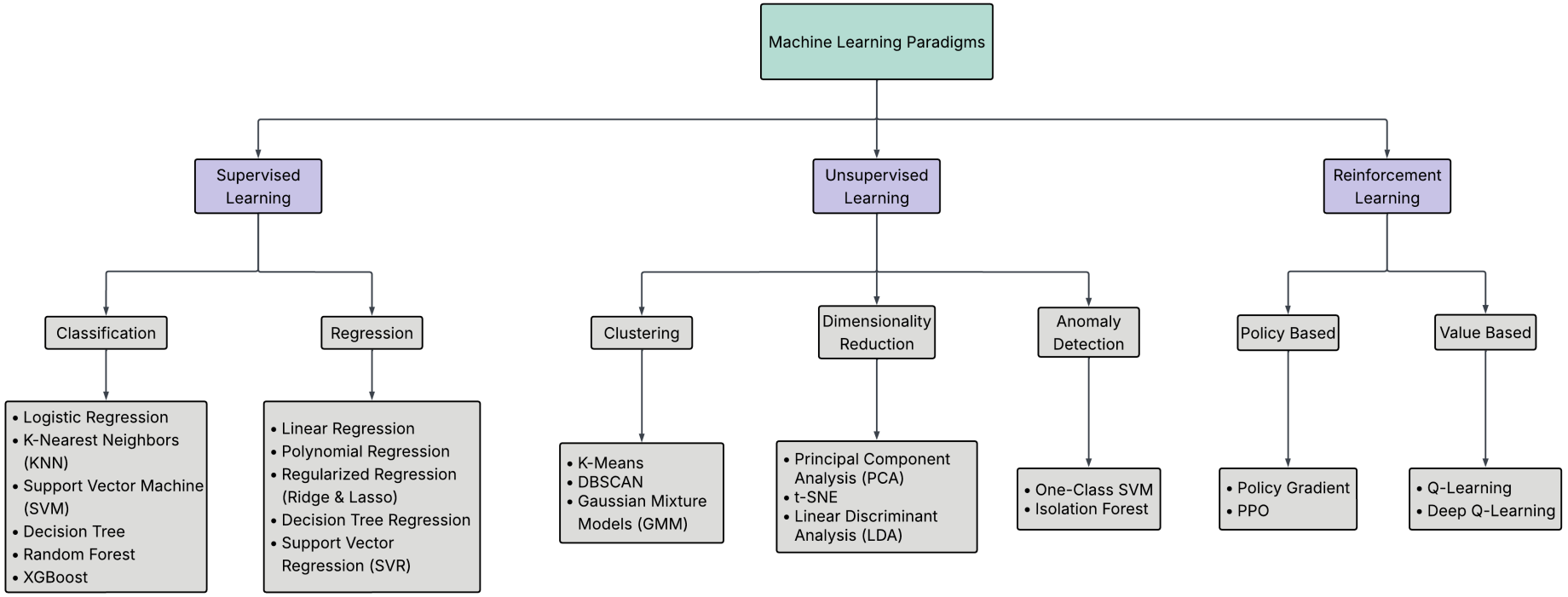


**Figure 2.1.2:** Overview of common machine learning methods categorised by paradigms and tasks, based on **sarker2021machine_learning_algorithms_real_world_applications_research_directions**, **geron2019hands_on_machine_learning**, and **zhang2020taxonomy_reinforcement_learning_algorithms**.

### 2.1.3 Neural Networks and Deep Learning

Neural networks and deep learning are often treated as a separate category in the machine learning literature. The following sections give a brief introduction to some of the most common neural network architectures and their typical use cases.

Neural networks are machine learning models inspired by the structure of the human brain, consisting of processing units organised in input, hidden, and output layers (**shrestha2019review_deep_learning_algorithms_architectures**). Deep learning refers to neural networks with many such layers. By stacking multiple layers, the network learns increasingly abstract representations of the input data, where lower layers detect simple patterns and higher layers combine these into more complex concepts (**lecun2015deep_learning**). This makes deep learning particularly well suited for complex, high-dimensional data such as images, audio, and text. (**lecun2015deep_learning**; **alzubaidi2021review_deep_learning_**
A practical limitation is that deep learning models generally require large amounts of training data to perform well (**alzubaidi2021review_deep_learning_concepts**).

#### Feedforward networks and MLP

The simplest deep learning architecture is the multilayer perceptron (MLP), also known as the feedforward neural network. In an MLP, information flows in one direction from input to output through one or more hidden layers, without any feedback connections (**shrestha2019review_deep_learning_algorithms_architectures**; **sarker2021machine_learning_algorithms_real_world_applications_research_dire**
Each node in one layer connects to each node in the following layer, and the network is trained using backpropagation to adjust the weights (**lecun2015deep_learning**).

#### Convolutional neural networks

Convolutional neural networks (CNNs) were originally developed for image recognition, where the spatial structure of the input is important (**lecun2015deep_learning**; **lecun1998cnn**). Rather than connecting every neuron to all neurons in the previous layer, CNNs use local connections and shared weights, which reduces the number of parameters and makes the network faster to train (**shrestha2019review_deep_learnin**
A typical CNN consists of convolutional layers, pooling layers, and fully connected layers. The convolutional layers progressively extract higher-level features from the input, while the pooling layers reduce the spatial size of the representations (**pouyanfar2018survey_deep_learning_algorithms**; **sarker2021machine_learning_a**
CNNs have achieved strong results in image recognition and computer vision, and

are also applied to other data types such as audio and text (**lecun2015deep_learning**; **pouyanfar2018survey_deep_learning_algorithms**).

### Recurrent neural networks and LSTM

Recurrent neural networks (RNNs) differ from feedforward networks in that the processing units form a cycle, allowing the network to maintain a state based on previous inputs (**shrestha2019review_deep_learning_algorithms_architectures**). This makes RNNs well suited for sequential data such as time series, speech, and text (**lecun2015deep_learning**; **pouyanfar2018survey_deep_learning_algorithms**). A known limitation of standard RNNs is the vanishing gradient problem, where information about earlier inputs is lost during training as the network processes longer sequences (**pouyanfar2018survey_deep_learning_algorithms**; **shrestha2019review_de**

Long short-term memory networks (LSTMs) address this by introducing memory cells and gating mechanisms that control what information is stored, read, and written (**hochreiter1997lstm**; **shrestha2019review_deep_learning_algorithms_architecture**). This allows LSTMs to learn from sequences with long-term dependencies, which makes them commonly used for time series analysis, speech recognition, and natural language processing (**lecun2015deep_learning**; **sarker2021machine_learning_algorithms_**

### Autoencoders

Autoencoders are neural networks trained to compress input data into a lower-dimensional representation and then reconstruct the original input from this representation (**shrestha2019review_deep_learning_algorithms_architectures**). Because they learn to reconstruct the input rather than predict a separate target, autoencoders are considered unsupervised models (**shrestha2019review_deep_learning_algorith**). They are commonly used for dimensionality reduction and anomaly detection, where observations that are difficult to reconstruct may indicate anomalies (**sarker2021machine_lear**; **shrestha2019review_deep_learning_algorithms_architectures**).

| Architecture | Description | Training Type | Common Applications |
|---|---|---|---|
| **CNN** (Convolutional Neural Network) | Uses convolutional layers to automatically learn spatial hierarchies of features from input data, especially images. | Supervised | • Image classification<br>• Object detection<br>• Image segmentation |
| **RNN** (Recurrent Neural Network) | Designed to handle sequential data by maintaining a hidden state that captures dependencies across time steps. | Supervised | • Sequence modeling<br>• Natural language processing<br>• Time series forecasting |
| **LSTM** (Long Short-Term Memory) | A type of RNN that uses gates and cell states to better capture long-term dependencies. | Supervised | • Natural language processing<br>• Speech recognition<br>• Time series prediction |
| **Autoencoder** (AE) | An unsupervised network that learns to encode input data into a lower-dimensional representation and then reconstruct it. | Unsupervised | • Dimensionality reduction<br>• Representation learning<br>• Anomaly detection |

**Figure 2.1.3:** Summary and comparison of common neural network architectures, based on **sarker2021machine_learning_algorithms_real_world_applications_research_dire**, **shrestha2019review_deep_learning_algorithms_architectures**, **pouyanfar2018survey_deep_learning_algorithms**, and **lecun2015deep_learning**.

### Deep reinforcement learning

Deep reinforcement learning (DRL) combines deep learning with reinforcement learning to build agents that can operate in high-dimensional environments. The main motivation for using deep networks in RL is to handle complex, high-dimensional state spaces that cannot be represented by simple tabular or linear methods (**zhang2020taxonomy_reinforcement_learning_algorithms**). A key development in DRL was the deep Q-network (DQN), which used a neural network to approximate the Q-value function and enabled agents to learn directly from raw inputs such as image pixels (**zhang2020taxonomy_reinforcement_learning_algo**). DRL has demonstrated strong performance in domains such as game playing, robotics, and control, where the state and action spaces are too large for traditional RL methods (**shrestha2019review_deep_learning_algorithms_architectures**; **zhang2020taxonomy_reinforcement_learning_algorithms**).

## 2.1.4 Attributes Relevant for Method Selection (MISSING)

Attributes relevant for method selection include:

- Data type: tabular, time series, image, text
- Data availability: large vs. limited data requirements
- Interpretability: interpretable vs. black-box methods

- Performance preferences: accuracy, training time, computational cost, prediction speed

### 2.1.5 Approaches to Method Selection

Selecting an appropriate machine learning method is not straightforward. As discussed in Section 1.1, the large number of available algorithms, each with distinct strengths and limitations, creates a real challenge for practitioners. This challenge is formally described by the Algorithm Selection Problem (**smith_miles2009algorithm_selection**), which asks which algorithm is likely to perform best for a given problem. The No Free Lunch theorems provide a theoretical basis for this difficulty, showing that no single algorithm performs best across all problems and that good selection requires understanding the characteristics of the problem at hand (**wolpert1997no_free_lunch**).

Several practical tools have been developed to help with method selection. Scikit-learn provides a publicly available decision flowchart that guides users through questions about data size and task type to suggest candidate algorithms (**sklearn_algorithm_cheatsh** and Microsoft Azure offers a similar cheat sheet for its machine learning platform (**azure_algorithm_cheatsheet**). While these tools are accessible and widely used, they have clear limitations. The selection criteria used in both tools are mainly accuracy and prediction speed, something that can in practice lead to poor results (**sala2018selecting_ml_algorithm**). Factors such as interpretability, data characteristics, and domain context are not considered, nor do they offer any guidance on how the problem itself should be formulated (**chen2023ml_manufacturing_industry4**).

More structured frameworks have been proposed in the academic literature. **sala2018selecting_ml_a** present a two-layer selection framework that incorporates algorithm characteristics such as scalability and robustness to outliers, in addition to learning type and response type. Domain-specific frameworks have also been proposed, such as that of **arena2024conceptual_framework_ml_algorithm_selection_predictive_maintenance**, who develop a structured set of decision variables for algorithm selection in predictive maintenance. However, as the authors note, most existing approaches focus narrowly on accuracy and computational requirements, and none provides a tool that connects algorithmic choices to the broader context of the application domain.

A related class of approaches is automated machine learning (AutoML), which seeks to automatically select, compose, and parametrise machine learning models to achieve optimal performance on a given task or dataset (**waring2020automated_machine_learni** AutoML systems automate parts of the machine learning pipeline, including data

preparation, feature engineering, hyperparameter optimisation, and model generation, with the goal of reducing the demand for expert knowledge and making machine learning more accessible to domain experts (**he2021automl_survey_state_of_the_a**
Within AutoML, the algorithm selection problem is typically treated as an optimisation task, where dataset characteristics are used to predict which algorithm is likely to perform best (**barbudo2023eight_years_automl_categorisation_review_tren**

However, AutoML approaches have notable limitations in the context of decision support for non-experts. Most systems treat the selection and configuration process as a black box, providing little explanation of why a particular configuration was chosen (**barbudo2023eight_years_automl_categorisation_review_trends**). This lack of transparency is a significant barrier to adoption, as users who do not understand or trust the output of a system are unlikely to apply it in practice (**waring2020automated_machine_learning_review_state_of_the_art_opportunit**
Furthermore, AutoML primarily addresses the later stages of the machine learning pipeline, and the phases considered inherently human, such as problem formulation and interpretation of results, have received little attention (**barbudo2023eight_years_autom**
This means that AutoML does not address the earlier challenge of helping practitioners understand which type of problem they have and which general approach is appropriate. This gap motivates the work in this thesis.

**Burde siste setning om "this gap" stå her? Eller ta med noe lignende i motivasjon? Eller ingen av delene?**

## 2.2 Machine Learning in Product Development and Production

### 2.2.1 Applications of Machine Learning

Machine learning is widely applied in industrial product development and production, where it supports tasks such as process optimisation, quality prediction, fault diagnosis, predictive maintenance, and anomaly detection (**wuest2016ml_manufacturing**; **plathottam2023review_ai_manufacturing**). The application of ML in these domains continues to grow, driven by the increasing availability of production data and the improved usability of ML tools (**wuest2016ml_manufacturing**). Unlike traditional rule-based approaches, ML methods can learn directly from data and adapt to changing conditions, making them well suited to the complex and dynamic nature of manufacturing environments (**wuest2016ml_manufacturing**). In practice, ML is used both to improve product quality and to optimise production processes, for example through early prediction of manufacturing outcomes, root

cause analysis, and condition monitoring of equipment (**weichert2019review_ml_optimization_pr**
As well as individual processes, ML also supports broader operational tasks such as production planning, scheduling, and supply chain management (**plathottam2023review_ai_manuf**
**presciuttini2024ml_iot_manufacturing_operations**).

### 2.2.2 Learning Paradigms in Industrial Applications

Across reported applications in manufacturing and production, supervised learning is the dominant paradigm, accounting for the large majority of studies in most application areas (**wuest2016ml_manufacturing**; **adekoya2022applications_ai_em**). This reflects the nature of many industrial problems, where labeled data are available through quality inspections, sensor logs, and operator feedback, and where the goal is typically to predict a known outcome such as product quality or equipment failure (**wuest2016ml_manufacturing**; **presciuttini2024ml_iot_manufacturing_operations**). Unsupervised learning plays a smaller but important role, particularly in anomaly detection and condition monitoring, where labeled examples of faults are scarce or costly to obtain (**presciuttini2024ml_iot_manufacturing_operations**). Reinforcement learning is less commonly reported in industrial settings, though it has shown strong results in process control and scheduling problems, where decisions must be made sequentially over time (**panzer2022deep_reinforcement_learning_productio**
In practice, the choice of learning paradigm is often shaped by what data is available rather than by the structure of the problem itself (**wuest2016ml_manufacturing**).

### 2.2.3 Pre-Master Findings

In a preceding specialization project, a large-scale analysis of machine learning research in product development and production was conducted, classifying over 33,000 article abstracts across product lifecycle phase, application area, learning paradigm, and reported ML methods. The results show that the literature is strongly concentrated in the optimisation phase of the product lifecycle, while early phases such as planning and concept development contain comparatively few studies. Supervised learning dominates across all phases and application areas, accounting for approximately 73–80% of the articles in each lifecycle phase, with neural networks, random forests, and support vector machines as the most frequently reported methods. Based on these findings, a roadmap was proposed that structures the literature across lifecycle phase, application area, and learning paradigm to support early-stage method selection (**bergsto2026mlselection**).

## 2.3 Ontologies and Knowledge Representation

### 2.3.1 Ontology Description

An ontology is an explicit specification of a conceptualization, where a conceptualization is an abstract model of a domain that identifies its relevant concepts and the relationships between them (**gruber1995toward_principles_design_ontologies_used_k** **studer1998knowledge_engineering_principles_methods** refine this by describing an ontology as a *formal, explicit specification of a shared conceptualization*. Each term in this definition carries a distinct meaning. *Formal* means the ontology is machine-readable and expressed in a language with precise semantics. *Explicit* means that all concepts and the constraints on their use are clearly defined, rather than left implicit in code or documentation. *Shared* means the ontology captures knowledge that is accepted by a group, rather than reflecting the perspective of a single individual (**studer1998knowledge_engineering_principles_methods**). In practical terms, an ontology defines a common vocabulary for a domain, including machine-interpretable definitions of its basic concepts and the relations among them (**noy2001ontology_development_101_guide_creating_first_ontology**).

An ontology consists of classes, which represent concepts in the domain, properties that describe features of those concepts, and relations that link concepts to one another (**noy2001ontology_development_101_guide_creating_first_ontology**). Classes are typically organised in a hierarchy where subclasses inherit the properties of their parent classes. Taken together, these elements allow a domain to be described in a way that both humans and software can interpret and reason over (**gruber1995toward_principles_design_ontologies_used_knowledge_sharing**).

The design of an ontology involves several practical constraints. **gruber1995toward_principle** argues that an ontology should require only the minimal ontological commitment needed for its intended purpose, meaning it should make as few claims about the world as necessary. This principle reflects a trade-off identified by **studer1998knowledge_engineering_principles_methods**: the more reusable an ontology is, the less directly applicable it tends to be in practice, since a more general representation provides less operational guidance for a specific task. **noy2001ontology_development_101_guide_creating_first_ontology** point to a related constraint on scope. An ontology should not attempt to capture everything that could be said about a domain, but only what is needed for the application at hand. Expanding scope beyond what the application requires adds complexity without benefit, and may introduce inconsistencies that are difficult to resolve. Ontology development is also not a one-time activity. As domain knowledge evolves and new requirements emerge, the ontology must be

revised, which means the initial design should anticipate extension and allow the vocabulary to be specialised without requiring changes to existing definitions (**gruber1995toward_principles_design_ontologies_used_knowledge_sharing**; **noy2001ontology_development_101_guide_creating_first_ontology**).

### 2.3.2 Why ML and Ontology

The machine learning domain has several properties that make ontological representation a natural fit.

ML methods are not a flat collection of independent techniques. They form hierarchical structures where more specific methods are specialisations of more general ones. **humm2026industry_ready_machine_learning_ontology** illustrates this directly: a multilayer perceptron is a specialisation of an artificial neural network, and if the latter can be used for classification and regression, the former inherits that applicability without needing it to be stated separately. This kind of reasoning over hierarchical structure is something an ontology supports naturally through inheritance, but is difficult to achieve in a flat list or a conventional database schema.

Beyond hierarchical structure, ML concepts are connected through typed semantic relations. Methods are suited to certain tasks and datatypes, belong to certain learning paradigms, and have different performance characteristics. These associations are not incidental but reflect the structure of the domain itself, and making them explicit is a prerequisite for structured method selection. **keet2015data_mining_optimization_ontology** motivate their ontology for data mining precisely on these grounds: traditional approaches treated learning algorithms as black boxes, correlating observed performance with data characteristics alone, while an ontology allows the internal characteristics of algorithms, their assumptions, optimisation strategies, and decision mechanisms, to be represented explicitly and reasoned over.

### 2.3.3 Semantic Web Standards

Ontologies, both in the ML domain and in general, are commonly implemented using standardised languages from the Semantic Web, a set of specifications developed by the World Wide Web Consortium (W3C) to make information machine-readable and interoperable across systems (**w3c**).

The foundation is the Resource Description Framework (RDF), the standard knowledge representation language for the Semantic Web (**gibbins2009resource_description_fram**

RDF represents knowledge as triples, each consisting of a subject, a predicate, and an object, forming a directed graph of binary relationships (**gibbins2009resource_description**
RDF Schema (RDFS) extends RDF with basic constructs for defining classes, properties, and constraints (**gibbins2009resource_description_framework_rdf**). Building on RDF, SKOS (Simple Knowledge Organization System) provides a lightweight standard for representing taxonomies and controlled vocabularies, without the formal reasoning capabilities of a full ontology language (**humm2026industry_ready_**

The Web Ontology Language (OWL) builds on RDF and RDFS and adds richer modelling constructs, including logical operators, quantifiers, and property characteristics such as transitivity (**bechhofer2009owl_web_ontology_language**). This gives OWL well-defined semantics that support automated reasoning, making it suitable for ontologies where logical consistency and entailment are required (**bechhofer2009owl_web_ontology_language**).

SPARQL (SPARQL Protocol and RDF Query Language) is used to query RDF data. It has syntax similar to SQL and retrieves data by matching triple patterns against stored graphs (**grobe2009rdf_jena_sparql_semantic_web**). RDF data is typically stored in triplestores, database systems designed specifically for handling RDF triples (**grobe2009rdf_jena_sparql_semantic_web**).

### 2.3.4 Existing Machine Learning Ontologies

Several ontologies have been developed to represent knowledge in the ML and data mining domain, each with a different scope and purpose. The following is a selection of openly available ontologies identified within the ML domain.

ML-Schema (**publio2018ml_schema_exposing_semantics_machine_learning_schem**
developed by the W3C Machine Learning Schema Community Group, is a top-level schema for representing and exchanging metadata about ML algorithms, datasets, models, and experiments. Its primary goal is interoperability. It provides a shared reference vocabulary that other ML ontologies can be mapped to, reducing fragmentation across platforms. ML-Schema defines around 25 abstract classes but contains no populated instances, meaning it describes structure rather than ML content. It is intended as a foundation for more specific ontologies, not as a standalone knowledge base.

OntoDM-core (**panov2014ontology_core_data_mining_entities**) is a formal ontology for core data mining entities such as datasets, tasks, algorithms, and their executions. It is structured across three layers. A specification layer covers abstract definitions, an implementation layer covers algorithm implementations and parameters, and an application layer covers executions and workflows.

OntoDM-core is aligned with established upper-level ontologies, which enables its use across different application domains (**panov2014ontology_core_data_mining_entities**). It has been applied in areas such as bioinformatics and drug discovery, and serves as a mid-level ontology for Exposé (**vanschoren2012experiment_databases**), an ontology for ML experiments used in OpenML. With 760 classes and detailed taxonomies of tasks and algorithms, it provides high structural granularity, but contains few populated instances and does not cover named ML methods, metrics, or libraries.

The MEX Vocabulary (**esteves2015mexvoc**) is a lightweight format for exchanging provenance metadata about ML experiments. It is organised in three layers. MEX-Core handles experiment configuration and execution, MEX-Algorithm covers algorithm class and learning method, and MEX-Performance covers evaluation measures. MEX is designed for basic ML iterations only and does not aim to cover the full data mining domain. Preprocessing semantics, model internals, and named ML methods are outside its scope.

ML Ontology (**humm2026industry_ready_machine_learning_ontology**) is the most comprehensive of the four in terms of ML content. It covers ML areas, tasks, approaches, preprocessing methods, metrics, libraries, AutoML solutions, and configuration parameters, with around 700 individuals and 5000 RDF triples. Unlike the other ontologies, it is not designed for experiment annotation but for integrating different ML tools and frameworks through a shared vocabulary. It supports forward-chaining inference via SPARQL and includes built-in quality assurance checks.

| Ontology | Purpose | ML Coverage | Size | Representation | Year |
|---|---|---|---|---|---|
| **ML Ontology** | Shared ML vocabulary for tool integration | Tasks, approaches, metrics, libraries, AutoML solutions, config. items | ~10 classes, ~700 individuals, ~5000 triples | RDF/RDFS, SKOS, SPARQL, SHACL | **2026** |
| **ML-Schema** | Schema for ML experiment metadata and interoperability | Algorithms, tasks, datasets, models, evaluations. | ~25 classes, no instances | OWL/RDF | **2016** |
| **OntoDM-core** | Semantic annotation of DM algorithms and processes | DM tasks, algorithms, datasets, generalizations, workflows | 760 classes, 221 individuals, 1419 axioms | OWL-DL | **2014** |
| **MEX Vocabulary** | Lightweight format for ML experiment metadata | Algorithm class, learning method, hyperparameters, performance measures | ~402 classes | RDF, PROV-O | **2015** |

**Figure 2.3.1:** Comparison of existing ML ontologies, adapted from **humm2026industry_ready_machine_learning_ontology**.

Across these ontologies, a distinction emerges between those more focused on experiment metadata and process structure (ML-Schema, OntoDM-core, MEX) and those focused on ML content and ML method knowledge (ML Ontology). The former are well suited for annotating and reproducing experiments, but provide little support for reasoning about which methods are applicable to a given problem. ML Ontology addresses this gap by representing the relationships between tasks, methods, and metrics in a form that can be queried and reasoned over.

### 2.3.5 ML Ontology Structure and Content

Among the ontologies reviewed, ML Ontology by **humm2026industry_ready_machine_learning_** is examined in more detail here because it provides the broadest coverage of ML concepts and methods, making it the most promising foundation for the system developed in this thesis.

ML Ontology is organised into five modules: classes, instances, inference rules, quality assurance checks, and predefined queries. Figure 2.3.2 shows this overall structure.

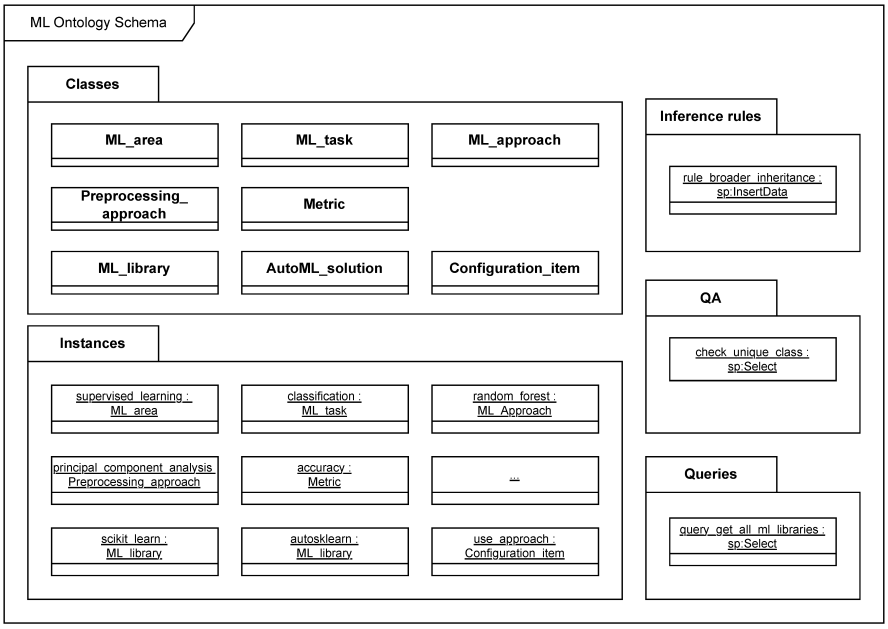


**Figure 2.3.2:** ML Ontology schema. Reproduced from **humm2026industry_ready_machine_learning_ontology**, licensed under CC BY 4.0.

The classes are divided into two groups: ML concepts, which cover the domain vocabulary, and ML implementations, which cover libraries, AutoML solutions, and their configuration options. Figure 2.3.3 shows the main classes and the relations between them.

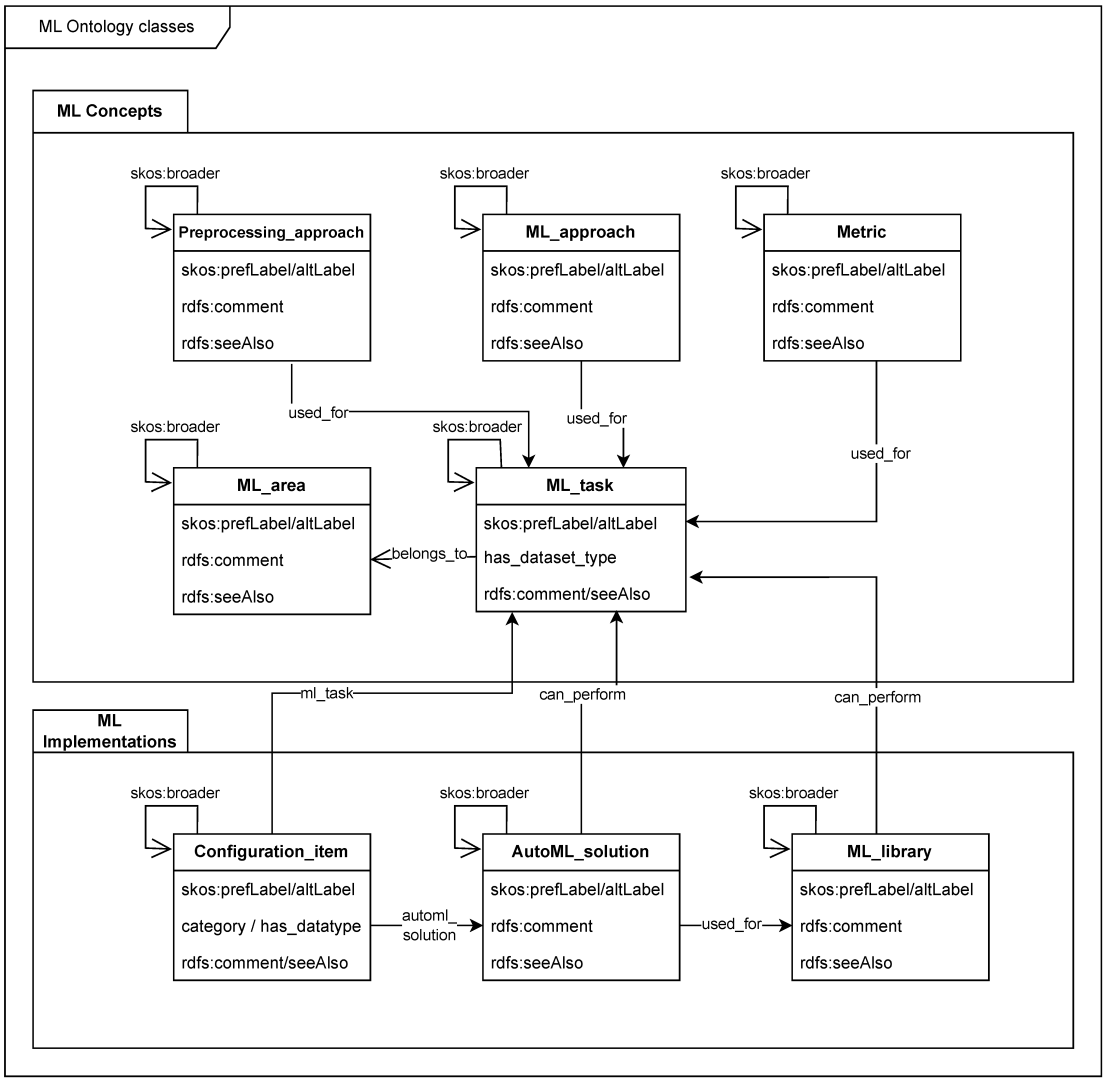


**Figure 2.3.3:** ML Ontology classes. Reproduced from **humm2026industry_ready_machine_learning_ontology**, licensed under CC BY 4.0.

Some key relations are presented in Figure 2.3.3, among them are `used_for`, which links approaches and metrics to the tasks they apply to, `belongs_to`, which links tasks to learning areas, and `can_perform`, which links libraries and AutoML solutions to tasks. Instances are also linked to corresponding entries in Wikidata (**wikidata**) via `rdfs:seeAlso`.

Since ML methods are central to the method selection task in this thesis, the properties of `ML_approach` instances are examined in more detail. Table 2.3.1 lists all properties associated with `ML_approach` instances in ML Ontology, as found in the published Turtle file (**humm2026industry_ready_machine_learning_ontology**). Table 2.3.1 lists the properties associated with `ML_approach` instances in ML Ontology.

**Table 2.3.1:** Properties of `ML_approach` instances in ML Ontology.

| Property | Description |
|---|---|
| `skos:prefLabel` | The preferred display name of the approach. |
| `skos:altLabel` | Alternative names or abbreviations, for example *ANN* for artificial neural network. |
| `skos:broader` | Links the approach to a more general parent approach in the hierarchy. |
| `rdfs:comment` | A short human-readable description of the approach. |
| `rdfs:seeAlso` | A link to an external resource, typically a Wikidata entry. |
| `used_for` | The ML tasks this approach can be applied to. |
| `performance` | Performance characteristics, for example *fast_training* or *accurate*. |
| `possible_if` | Conditions under which the approach is applicable, for example *numerical_features*. |
| `not_recommended_if` | Conditions under which the approach should be avoided, for example *many_features*. |

Coverage varies considerably across properties. Table 2.3.2 shows how many of the 122 `ML_approach` instances have each property populated.

**Table 2.3.2:** Coverage of `ML_approach` properties across 122 instances in ML Ontology.

| Property | Coverage |
|---|---|
| `skos:prefLabel` | 122/122 (100%) |
| `used_for` | 103/122 (84%) |
| `rdfs:comment` | 97/122 (79%) |
| `skos:altLabel` | 64/122 (52%) |
| `skos:broader` | 56/122 (45%) |
| `rdfs:seeAlso` | 56/122 (45%) |
| `performance` | 23/122 (18%) |
| `possible_if` | 9/122 (7%) |
| `not_recommended_if` | 3/122 (2%) |

The low coverage of some properties reflects that they must be populated explicitly for each instance. However, ML Ontology includes a forward-chaining inference rule that propagates selected properties along `skos:broader` hierarchies. Properties marked as *broader inherited* are automatically assigned to more specific concepts based on what is defined for their parent. For the properties in Ta-

ble 2.3.2, this applies to `used_for`, `belongs_to`, `performance`, `possible_if`, and `not_recommended_if`. The rule must be executed after loading the ontology, and is not applied automatically.

ML tasks are also organised in hierarchies using `skos:broader`. For tasks, the inference rule means that if `classification` belongs to `supervised_learning`, then `text_classification` inherits that relation without it needing to be stated explicitly. Each `ML_task` instance may also carry a `has_dataset_type` property that specifies which data types the task applies to, for example *tabular* or *image*. Since `ML_approach` instances are linked to tasks via `used_for`, dataset type is accessible indirectly through this relation. This means that an approach applicable to `image_classification` is implicitly associated with image data through the task it is linked to. Figure 2.3.4 illustrates the task hierarchy for supervised and unsupervised learning.

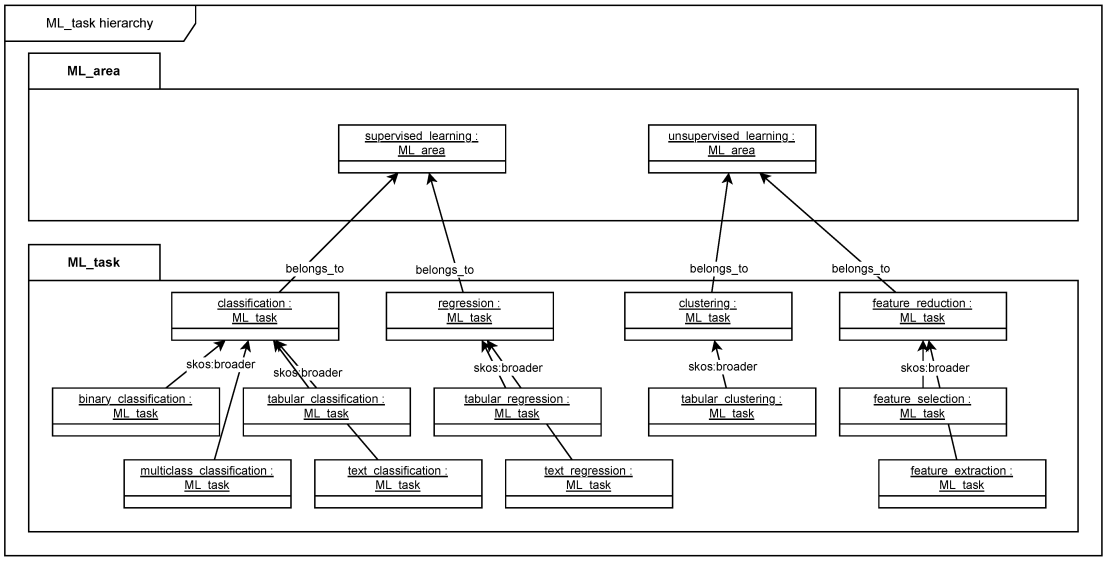


**Figure 2.3.4:** ML task hierarchy in ML Ontology. Reproduced from **humm2026industry_ready_machine_learning_ontology**, licensed under CC BY 4.0.

Reinforcement learning is defined as an ML area in the ontology but does not appear in the hierarchy since no tasks and no methods have been assigned to it. Coverage of `ML_approach` instances is also incomplete. **humm2026industry_ready_machine_l** report a recall of 0.84 for this class, with graph neural networks, variational autoencoders, and genetic algorithms identified as examples of missing concepts.

## 2.4 Semantic Retrieval and Text Similarity

Ontologies provide structured, explicit representations of domain knowledge, but they depend on manually defined concepts and relations. This makes them well-suited for reasoning over known entities, yet limited in their ability to handle descriptions expressed in natural language or to retrieve knowledge that was not explicitly encoded. Semantic retrieval addresses this gap by operating on the meaning of text rather than on predefined structure. This section introduces the core concepts underlying this approach: how text can be represented as numerical vectors, how similarity between texts can be measured in that representation, and how these techniques support ranking and matching against a corpus of scientific literature.

### 2.4.1 Text Embeddings and Semantic Similarity

In vector semantics, text is represented as a numerical vector, a point in a high-dimensional space referred to as the embedding space (**jurafsky_martin2026speech_language_pro**
The position of a vector in this space reflects the semantic content of the text it represents. A central property of this representation is that semantically similar texts are mapped to nearby points in the embedding space, while unrelated texts are placed further apart. Figure 2.4.1 illustrates this principle for a small set of example words.

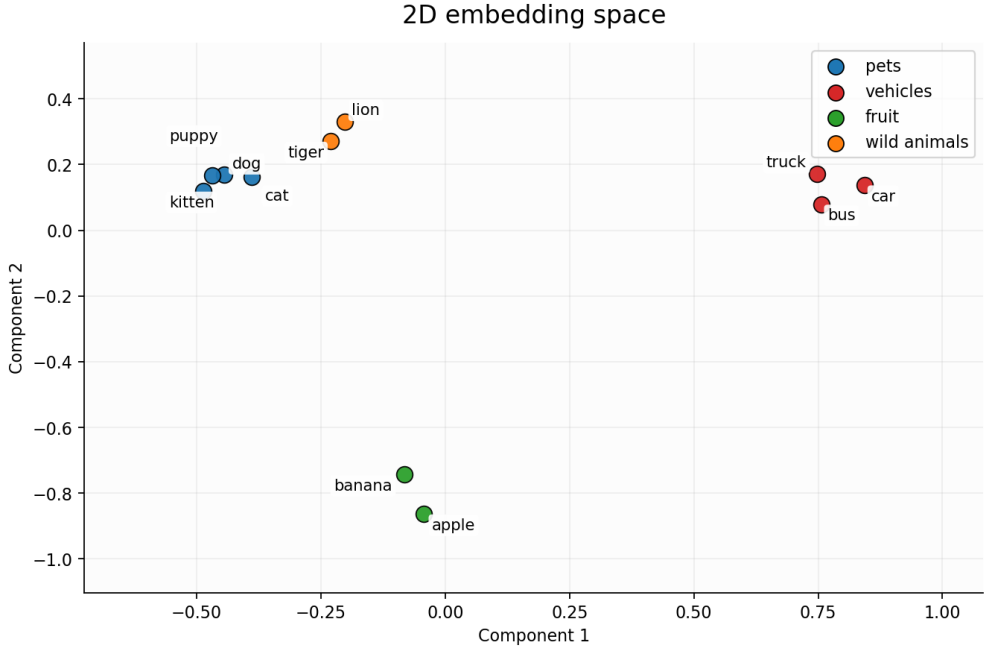


**Figure 2.4.1:** A two-dimensional projection of example word embeddings. Semantically similar words cluster together in the embedding space, while unrelated words are positioned further apart.

To compare two vectors $\mathbf{u}$ and $\mathbf{v}$, cosine similarity is the most common metric used in natural language processing (**jurafsky_martin2026speech_language_processing**). Rather than measuring absolute distance between two points, it measures the angle between two vectors, normalised for their length:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|} \tag{2.1}$$

The result ranges from 1, indicating identical direction and maximum similarity, to 0 for orthogonal vectors with no similarity (**jurafsky_martin2026speech_language_proces** Normalising for vector length ensures that the similarity score reflects semantic content rather than vector magnitude. In practice, cosine distance, defined as $1 - \cos(\mathbf{u}, \mathbf{v})$, is often used when a distance metric is required, where lower values indicate greater similarity.

### 2.4.2 Transformer-Based Sentence Embedding Models

Transformer-based language models represent text by processing input tokens through multiple layers of self-attention, producing contextual representations that capture relationships between words across the full input sequence (**vaswani2023attentionneed** Unlike earlier embedding approaches that assigned a fixed vector to each word regardless of context, transformer models produce representations that reflect the meaning of a word given its surrounding text.

However, standard transformer models such as BERT are not directly suited for computing sentence-level similarity. As noted by **reimers2019sentence_bert_sentence_em** comparing two sentences with BERT requires feeding both into the network simultaneously, which becomes computationally infeasible at scale. Finding the most similar pair in a collection of 10,000 sentences would require approximately 50 million inference computations.

Sentence-BERT (SBERT) addresses this by fine-tuning BERT using a siamese network structure, producing fixed-size sentence embeddings that can be compared efficiently using cosine similarity (**reimers2019sentence_bert_sentence_embeddings_usin** The key property is that semantically similar sentences are placed close together in the resulting embedding space, making SBERT suitable for tasks where embeddings for a large corpus can be precomputed and a new query compared against all of them in milliseconds.

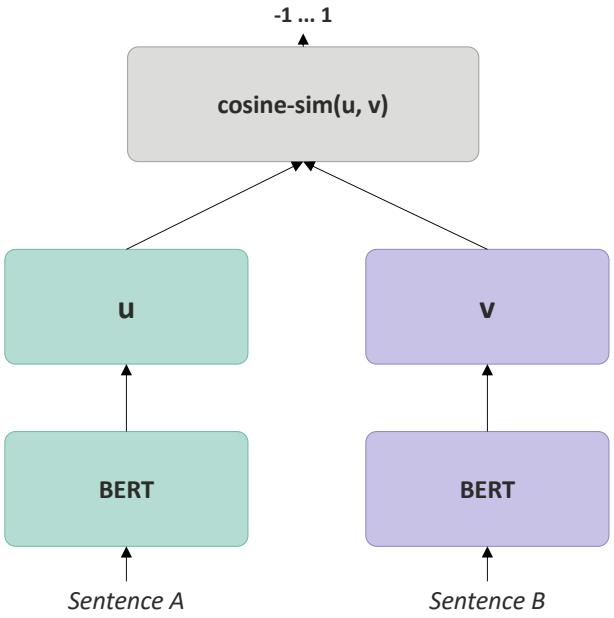


**Figure 2.4.2:** SBERT architecture at inference time. Each sentence is encoded independently through a shared BERT network, producing vectors **u** and **v** that are compared using cosine similarity. Reproduced from **reimers2019sentence_bert_sentence_embeddings_using_siamese_bert_networks** under CC BY-SA 4.0.

The sentence-transformers library provides pretrained models based on this approach for encoding text into fixed-size vectors (**sbert_docs**).

### 2.4.3 Semantic Retrieval

**Flere kilder her!**

Information retrieval (IR) is the task of identifying and ranking documents from a collection according to their relevance to a user query (**jurafsky_martin2026speech_language_pro**
Traditional approaches represent both documents and queries as sparse vectors of word counts, weighted by schemes such as tf-idf or BM25, and compute similarity based on shared terms (**jurafsky_martin2026speech_language_processing**). While effective for many tasks, this approach treats documents as bags of words, meaning that word order and meaning are ignored, and only exact term matches contribute to the relevance score.

Dense retrieval addresses this limitation by representing documents and queries as embeddings computed from language models, rather than as word count vectors (**jurafsky_martin2026speech_language_processing**). Because semantically similar texts are mapped to nearby points in the embedding space, a query

can retrieve relevant documents even when they do not share exact terms with the query. This makes dense retrieval more flexible than keyword-based approaches for tasks where the user's information need is expressed in natural language rather than precise search terms.

Figure 2.4.3 illustrates the distinction with a constructed example. The same query returns only Document A under keyword-based retrieval, as it is the only document sharing exact terms with the query. Dense retrieval ranks all three documents by semantic similarity, including Documents B and C despite the absence of exact term overlap in these documents.

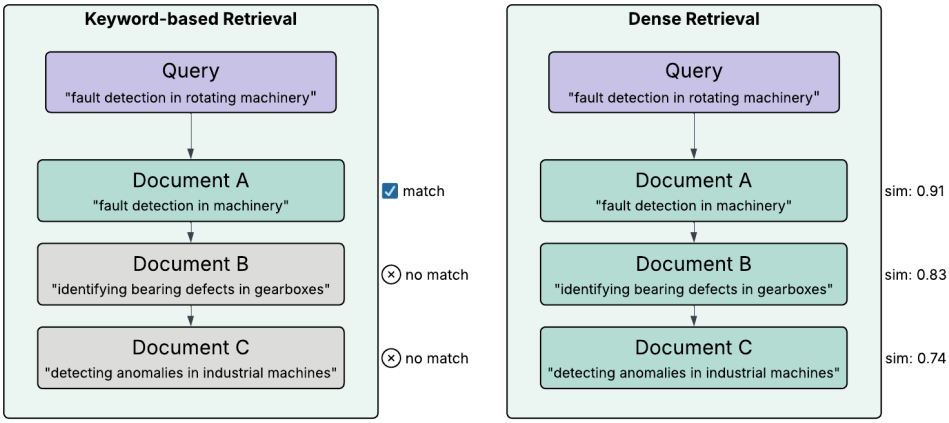


**Figure 2.4.3:** Keyword-based versus dense retrieval for the same query.

### 2.4.4 Cross-Encoder Reranking

The sentence embedding approach described in Section 2.4.2 is an example of a bi-encoder, where query and document are encoded independently into fixed-size vectors and similarity is measured using cosine similarity (**reimers2019sentence_bert_sentence**
Because the representations are computed independently, document embeddings can be precomputed and stored, making bi-encoders efficient for large-scale retrieval (**thakur2021augmented_sbert_data_augmentation_method_improving_bi_**

A cross-encoder takes a different approach, concatenating query and document into a single input and processing both jointly, which allows the model to apply attention across both texts simultaneously (**reimers2019sentence_bert_sentence_embeddings**
This produces a direct relevance score rather than a similarity between independent vectors. Cross-encoders are generally more accurate than bi-encoders, but since no independent representations are computed, every query-document pair must be processed from scratch at query time, making cross-encoders impractical for retrieval over large document collections (**humeau2020poly_encoders_transformer_ar**
**thakur2021augmented_sbert_data_augmentation_method_improving_bi_encod**

Figure 2.4.4 illustrates the architectural difference between the two approaches.

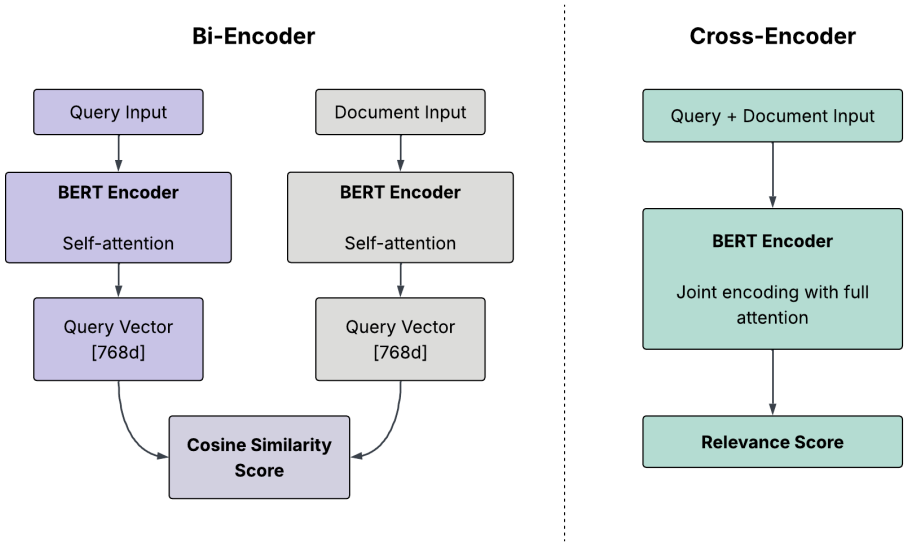


**Figure 2.4.4:** Bi-encoder and cross-encoder architectures. The bi-encoder encodes query and document independently, producing vectors that are compared using cosine similarity. The cross-encoder concatenates both inputs and processes them jointly, enabling full attention between query and document inputs and producing a direct relevance score. Inspired by **kumar2024crossencoders**.

A common approach to combining the strengths of both architectures is a two-stage retrieval pipeline (**nogueira2019passage_re_ranking_with_bert**). In the first stage, a retrieval system narrows the full document collection down to a smaller set of candidates. In the second stage, a cross-encoder reranks these candidates by scoring each one against the query directly, producing a more precise relevance ranking. Bi-encoders are well suited for the first stage for the same reason described above: document embeddings can be precomputed, making large-scale candidate retrieval efficient (**thakur2021augmented_sbert_data_augmentation_method_impro**
**dorkin2025tartunlp_subject_tagging_two_stage_ir**; **verma2025beyond_retrieval_ensen**

This pipeline has been applied across diverse retrieval tasks, including ranking text passages by relevance (**nogueira2019passage_re_ranking_with_bert**), biomedical question answering (**verma2025beyond_retrieval_ensembling_cross_encoders**), and subject tagging (**dorkin2025tartunlp_subject_tagging_two_stage_ir**). Cross-attentional reranking has been shown to generalise well across diverse retrieval tasks (**thakur2021beir_heterogenous_benchmark_zero_shot_evaluation_ir_mode**
and in several settings the two-stage pipeline has shown substantially better retrieval performance than using a bi-encoder alone (**dorkin2025tartunlp_subject_tagging_two_s**
**verma2025beyond_retrieval_ensembling_cross_encoders**).

### 2.4.5 Scientific Literature as a Knowledge Source

Scientific articles document how machine learning methods have been applied across a wide range of problems and contexts. In the domain of machine learning for engineering and manufacturing, a large number of such articles exist, covering diverse algorithms and applications (**sala2018selecting_ml_algorithm**). This breadth of literature is itself a challenge. Relevant knowledge is spread across journals, conference proceedings, and research communities (**adekoya2022applications_ai_em**), and the volume of available work makes it difficult for any individual to draw connections across topics and domains (**olivetti2020data_driven_materials_research_enabl**). The result is that the literature creates extensive knowledge on the topic, while at the same time making the selection of the right approach difficult (**sala2018selecting_ml_algo**).

Semantic retrieval offers a way to address this challenge by enabling systematic matching between a problem description and relevant articles. Article abstracts are a practical textual basis for this purpose. Text mining of scientific literature has mostly been carried out on abstracts due to their availability, and the fact that abstracts consist of shorter sentences presenting only the most important findings of a study (**westergaard2018comprehensive_quantitative_comparison_text_mining_15**). While full-text articles contain richer information, they also include complex tables, references, and speculative discussion sections, making them less consistent as a basis for comparison across a large corpus (**westergaard2018comprehensive_quantitative**). Abstracts thus represent a structured and consistently available representation of each article's core content.

## 2.5 User Feedback in Recommendation Systems

Recommender systems collect information on user preferences, either explicitly through numerical ratings, or implicitly by monitoring user behaviour (**bobadilla2013recomme**). This feedback can be stored and used to improve recommendations in subsequent interactions, so that the longer users engage with the system, the more the output can be refined to match their preferences (**ricci2010recommender_systems_handbook**).

A practical problem with explicit ratings is that some items receive few ratings, making their average score unreliable. This is related to the sparse data problem, which frequently occurs when estimating counts from small amounts of data (**murphy2012machine_learning_probabilistic_perspective**). Bayesian smoothing addresses this by pulling the estimated score towards a neutral prior when little data is available. The posterior is a compromise between what was previously believed and what the data indicates (**murphy2012machine_learning_probabilistic_perspe**).

As more data accumulates, the observed average comes to dominate the estimate, since the data has overwhelmed the prior (**murphy2012machine_learning_probabilistic_perspec**

When a rating-based signal must be combined with a score from a separate ranking process, the two may not be directly comparable in scale. Reciprocal Rank Fusion addresses this by converting rank positions into scores rather than using the underlying score values directly. Highly-ranked items receive higher scores, but the importance of lower-ranked items does not vanish entirely, as it would with an exponential function (**cormack2009reciprocal_rank_fusion**). This approach combines ranks without regard to the arbitrary scores returned by particular ranking methods, making it robust to differences in scale between sources (**cormack2009reciprocal_rank_fusion**).

The effectiveness of explicit feedback depends on having sufficient ratings. The cold start problem occurs when it is not possible to make reliable recommendations due to an initial lack of ratings (**bobadilla2013recommender_systems_survey**). A particular variant is the new community problem, which refers to the difficulty of obtaining a sufficient amount of rating data when a system is first deployed (**bobadilla2013recommender_systems_survey**). Under these conditions, enough ratings have to be collected before the system can understand user preferences and provide reliable recommendations (**ricci2010recommender_systems_handbook**).

## 2.6 Related Work and Existing Tools

Few existing tools directly address the combination of method selection, knowledge representation, and implementation support that this thesis investigates. Most related tools either focus on automating a specific part of the ML workflow or provide broad access to AI resources without guiding users through choosing a suitable method for their problem. The following sections review the most relevant examples found.

### 2.6.1 Ontology-Based Meta AutoML

OMA-ML (**zender2022ontology_based_meta_automl**) is a system that combines multiple AutoML solutions through the ML Ontology, described in Section 2.3.5. The ML Ontology is used to filter configuration options and select appropriate AutoML strategies, making the process more transparent than standard AutoML systems. It also supports a wider range of methods by integrating solutions across multiple ML libraries rather than being tied to a single one. Like

other AutoML tools, OMA-ML requires the user to provide a labeled dataset and define the ML task upfront.

### 2.6.2 Template-Based Code Generation

Traingenerator (**rieke2021traingenerator**) is a web-based tool that generates template Python code and Jupyter notebooks for ML tasks. Users select a task and framework from a menu and receive a working code template covering common steps such as data loading, model setup, training, and evaluation. The generated code can be downloaded as a Python script or notebook, and optionally opened directly in Google Colab.

The tool assumes the user has already decided on a method and framework before use. At the time of writing, the hosted version of Traingenerator produces a runtime error and does not render output correctly, likely due to an outdated Streamlit API call. The tool appeared to function as intended during the development period of this thesis.

### 2.6.3 AI-on-Demand Platform

The European AI-on-Demand Platform (AIoDP) and its DeployAI implementation offer a broad marketplace of pre-trained models, reusable AI modules, and domain-specific applications supported by cloud and HPC infrastructure (**troumpoukis2025dep**
The platform targets SMEs, researchers, and public sector organisations and provides tools for building, training, and deploying AI solutions at scale. Users can browse and filter available solutions to narrow down options relevant to their domain or use case. The primary focus is on access and deployment of existing solutions rather than on guiding users through method selection for a new problem.

CHAPTER

# THREE

# METHODOLOGY AND SYSTEM OVERVIEW

This chapter presents the methodological framework and describes the design and development of MLguide, a web-based decision-support system for machine learning method selection. Section 3.1 introduces Design Science Research as the overarching framework. Section 3.2 gives a high-level overview of MLguide and its main components. The following sections describe the design goals, the development process, and the user testing approach.

## 3.1 Design Science Research

This thesis uses Design Science Research (DSR) as its methodological framework, following **peffers2007design_science_research_methodology** and **hevner2004design_s**
DSR is a well-established approach for developing and evaluating decision support systems (**arnott2012design_science_decision_support_systems_research**), and has been applied in recent work combining machine learning with user-facing decision support (**landwehr2022design_knowledge_deep_learning**; **dellermann2018des**
In DSR, knowledge is gained by building and evaluating an artifact, producing both a usable system and knowledge that can inform future work of a similar kind (**landwehr2022design_knowledge_deep_learning**; **hevner2004design_science_infor**
The DSR approach is therefore suited for this work, where the research questions are investigated through the design and evaluation of the artifact, MLguide.

The DSR process consists of six activities: problem identification, defining objectives, design and development, demonstration, evaluation, and communication (**peffers2007design_science_research_methodology**). These activities are

nominally sequential, but iteration is expected. After evaluation, the researcher may return to design and development to improve the artifact before proceeding (**peffers2007design_science_research_methodology**). Figure 3.1.1 shows how these activities map to the structure of this thesis.

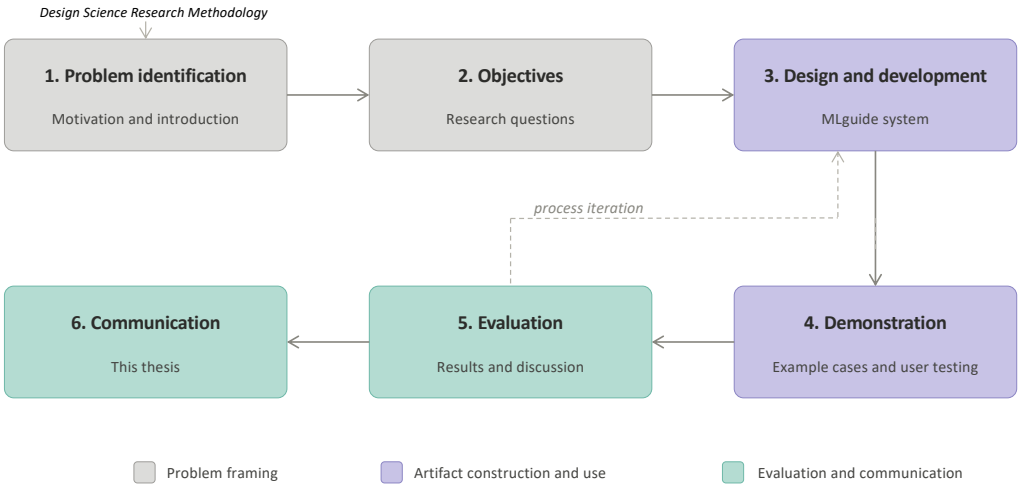


**Figure 3.1.1:** Design Science Research Methodology adapted from **peffers2007design_science_research_methodology**.

The process in this thesis followed a design-centered initiation, where development of the system started before the research questions were fully formalised (**peffers2007design_science_research_methodology**).

## 3.2 System Overview

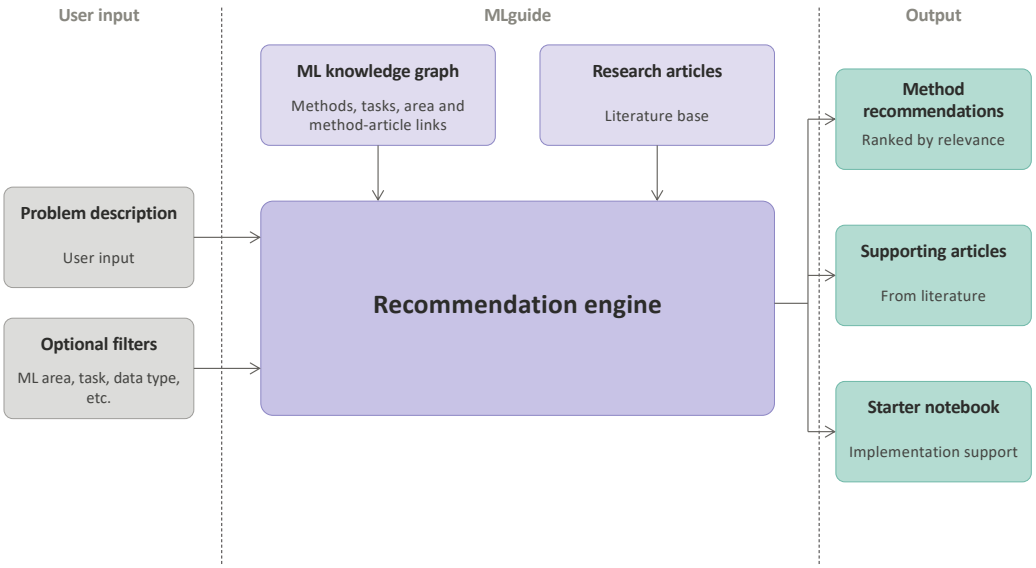


**Figure 3.2.1:** System Overview of MLguide.

Figure 3.2.1 gives a high-level overview of MLguide and its main components. MLguide is a web-based decision-support system designed to help users select machine learning methods for a given problem. The user starts by describing their problem with text input, and can optionally provide filters to narrow the scope of the problem, such as ML area (earlier referred to as paradigm in this thesis) and task type.

The system uses two knowledge sources. The first is a structured ML knowledge graph containing information about machine learning methods and tasks, including links between methods and the research articles they appear in, among other things. The second is a database of research articles from the literature. With the help of these knowledge sources, the recommendation engine produces three outputs: a ranked list of ML methods, a set of supporting articles, and a starter notebook, a Jupyter notebook file with code generated by MLguide as a starting point for implementation.

## 3.3 Design Goals and Motivation

MLguide was developed as a direct continuation of the work done in the pre-master project (**bergsto2026mlselection**). The developed roadmap outlines a process for filtering and ranking research literature based on user-defined context parameters, including ML area, product lifecycle (PLC) phase, and application

area. This is illustrated with a simplified version of the roadmap in Figure 3.3.1. In the pre-master project, the main focus was on collecting and analysing research literature. Several directions for future development were identified, including better support for problem formulation, ontology-based ML knowledge representation, support for translating recommendations into an ML implementation, and incorporating user feedback to improve method recommendations over time (**bergsto2026mlselection**).

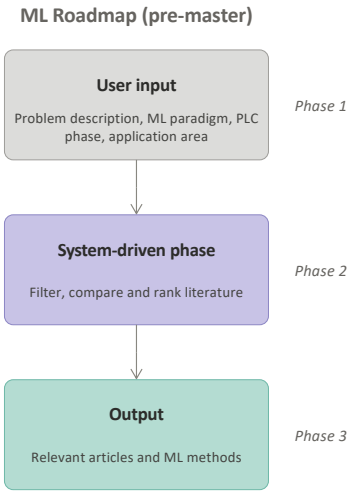


**Figure 3.3.1:** Simplified ML selection roadmap from the pre-master project (**bergsto2026mlselection**).

MLguide addresses these directions by extending the original roadmap into a web-based decision-support system. The main design goals are to support users in formulating their problem, to recommend relevant ML methods based on both structured knowledge and research literature, to provide a starting point for implementation through generated notebooks, and to incorporate user feedback on recommendations. The system is primarily aimed at engineers and students who are new to machine learning and need structured guidance in choosing a suitable method. A web-based interface was chosen to lower the threshold for use, making it easy for other users to access and test the system without installation.

## 3.4 Incremental and Iterative Development

MLguide was developed incrementally over the course of the master's project. New features and components were built and deployed one at a time, each adding to the working system. This approach is consistent with incremental software development, where the system grows through a series of versions rather than

being built all at once (**sommerville2011software_engineering**).

Figure 3.4.1 illustrates this process. Planning, development, and self-evaluation were carried out as concurrent activities, producing successive versions of the system. Throughout development, individual features and the overall system output were continuously tested to check that behaviour was correct and results were useful.

The development also had an iterative character. Feedback from the thesis supervisor informed adjustments to the system design and priorities throughout the development period. In addition, two rounds of user testing were conducted, and the findings were used to evaluate and adjust the system. This iterative cycle is consistent with the DSR process described by **peffers2007design_science_research_methodolog** where evaluation results inform a return to design and development. The user testing setup is described in the following section.

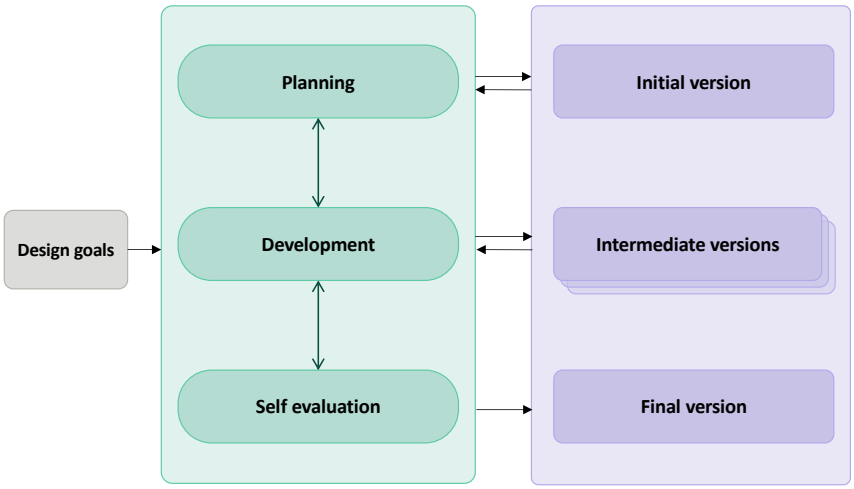


**Figure 3.4.1:** Illustration of the incremental development process, based on **sommerville2011software_engineering**.

## 3.5 User Testing

### 3.5.1 Setup and Context

User testing was conducted in collaboration with the course TMM4128 Machine Learning for Engineers at the Norwegian University of Science and Technology (NTNU). The course provided a natural opportunity to test the system with its intended user group. Students used MLguide as part of their course projects. The participants were engineering students with limited prior experience with machine learning. Feedback could be collected through a feedback form accessible

in MLguide and from the students' project reports. The two course projects are summarised in Table 3.5.1.

**Table 3.5.1:** Description of the two user testing sessions conducted in collaboration with the course TMM4128 Machine Learning for Engineers at NTNU.

| | **Project 1** | **Project 2** |
|---|---|---|
| **Topic** | Anomaly detection in time series data | Reinforcement learning in product lifecycle |
| **ML area** | Unsupervised learning | Reinforcement learning |
| **Task** | Detect anomalies using unsupervised learning techniques and compare performance | Analyse RL applications across lifecycle phases and implement a sustainability agent |
| **MLguide task** | Test the system on their problem and report on relevance and usefulness of results | Assess the applicability of MLguide to their problem |
| **Deadline** | March 2026 | April 2026 |

The two projects differed in scope and focus. The first project had a well-defined problem: detecting anomalies in time series data using unsupervised learning. Students were asked to compare at least three techniques, which made MLguide a natural fit. The system could be used early in the project to get method recommendations, or later as a comparison against methods already selected.

The second project was more open-ended. Students were asked to analyse applications of reinforcement learning across four product lifecycle phases: product design, manufacturing, product use, and recycling, and to implement a simulated RL environment and a sustainability agent. MLguide could be used in two ways in this context: to find relevant articles by submitting problem descriptions related to different lifecycle phases, and to get algorithm recommendations for the implementation part of the project.

A simple overview of the development timeline, including the user testing sessions, is illustrated in Figure 3.5.1.

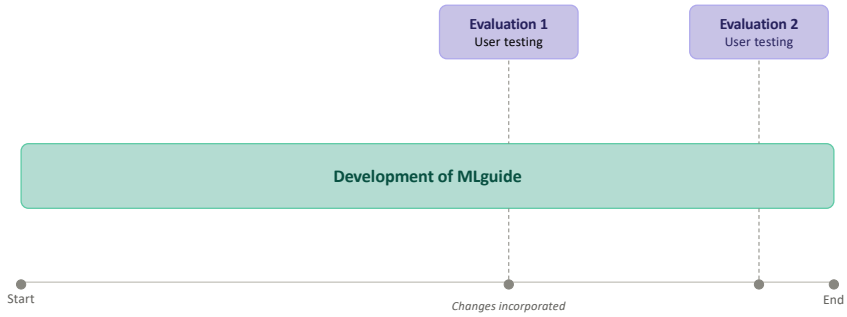


**Figure 3.5.1:** Development timeline of MLguide.

CHAPTER

# FOUR

# IMPLEMENTATION

This chapter describes the implementation of MLguide, a web-based decision-support system for machine learning method selection. The system combines an ontology-based knowledge graph with semantic retrieval from scientific literature to produce ranked method recommendations grounded in research literature. It also provides starter notebook templates to support early-stage implementation.

> The implementation of MLguide is published as open-source and is available at: `https://github.com/mbergsto/MLguide.git`

## 4.1 System Architecture

MLguide is structured as a three-tier web application consisting of a frontend, a backend, and a database layer, deployed as Docker containers (**docker**). Docker was chosen because containerisation makes the system easy to run across different environments without managing dependencies manually, which is important for a research artefact that needs to be shared and tested by others.

Figure 4.1.1 shows the main components and how they communicate. The frontend is implemented in Streamlit (**streamlit**), which was chosen because it allows interactive web applications to be built directly in Python, without writing HTML or JavaScript. This made it well suited for rapid development of a data-driven prototype. The frontend communicates with the backend through an HTTP API. The backend is implemented in FastAPI (**fastapi**), which provides automatic API documentation and built-in data validation, and contains the core application

logic.

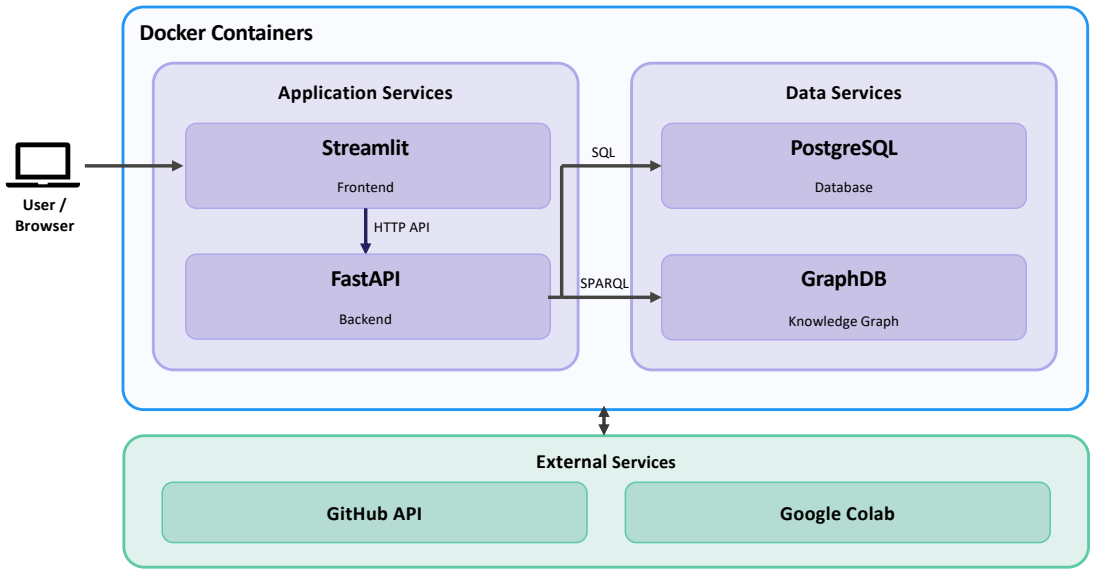


**Figure 4.1.1:** System Overview of MLguide.

The backend reads from two data stores. GraphDB (**graphdb**) stores the ML ontology and article links, and is queried using SPARQL. It was chosen because it is designed specifically for RDF data and supports SPARQL, making it a natural fit for an RDF-based knowledge graph. PostgreSQL (**postgresql**) stores article abstracts and vector embeddings using the pgvector extension (**pgvector**). This allows embeddings and relational data to be stored and queried in the same database, avoiding the need for a separate vector database. The frontend also integrates with two external services: the GitHub API and Google Colab, which are used to upload and open generated notebooks.

MLguide follows a layered architecture, in which each layer only depends on the layer immediately beneath it (**sommerville2011software_engineering**). This keeps concerns separated across the system, so that individual layers can be modified or replaced without affecting the rest of the application. The layered approach also supports incremental development, because each layer can be made available as it is completed without waiting for the full system to be built (**sommerville2011software_engineering**). This is consistent with the development process described in Section 3.4.

Figure 4.1.2 shows the internal layer structure of the frontend and backend. Both are organised into distinct layers with clear responsibilities. In the backend, the router layer handles incoming API requests and passes them to the service layer, which contains the application logic. The persistence layer is responsible for all

database access and isolates data retrieval from the rest of the application. In the frontend, the presentation layer handles rendering and user interaction. The application logic layer coordinates calls to the backend and external services. Because the frontend is a Streamlit application that delegates most logic to the backend, the majority of modules in this layer are thin wrappers around the integrations layer. The clearest exceptions are the notebook template services, which contain local logic for selecting and rendering starter notebooks.

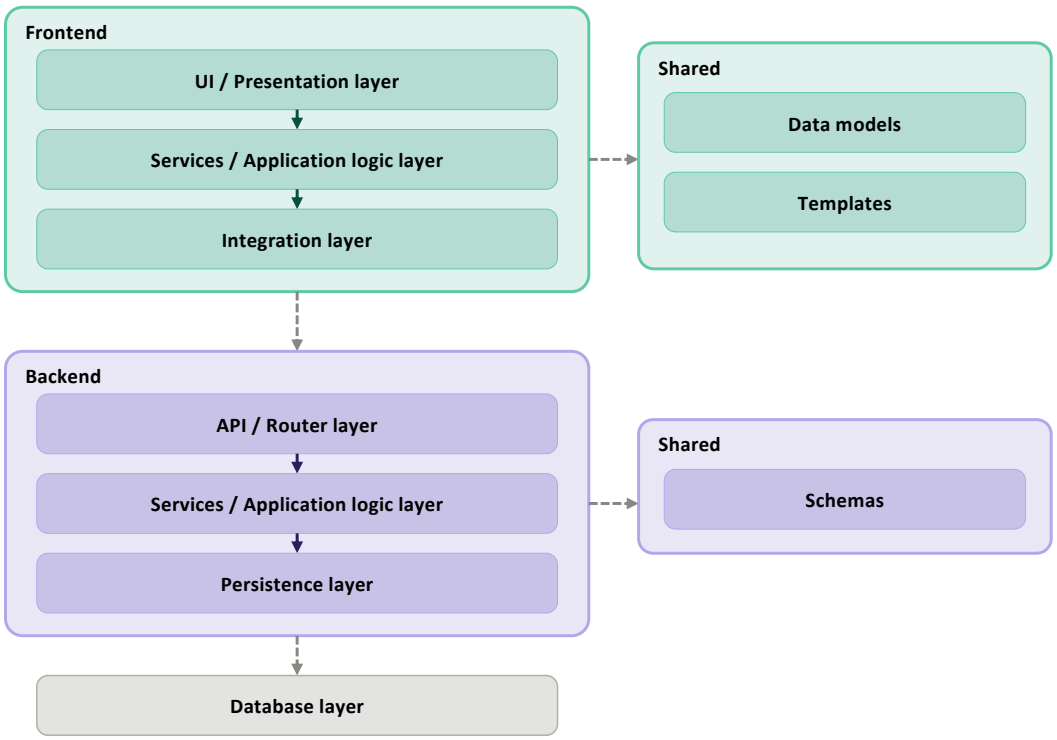


**Figure 4.1.2:** Layered Architecture of MLguide.

Figure 4.1.3 shows the full package structure of both the frontend and backend, including the main modules within each layer. In addition to the recommendation logic, the system handles two other concerns that run through most layers. User services is a lightweight feature covering authentication and saved searches, allowing users to store and reload previous searches with problem descriptions. Feedback handling allows users to rate recommended methods, with ratings stored in PostgreSQL and used to adjust method ranking over time. Both concerns follow the same layered structure as the recommendation logic, with separate modules in the router, service, and persistence layers of the backend, and corresponding wrappers in the frontend application logic layer. The feedback logic is further described as part of the recommendation process in Section 4.3.

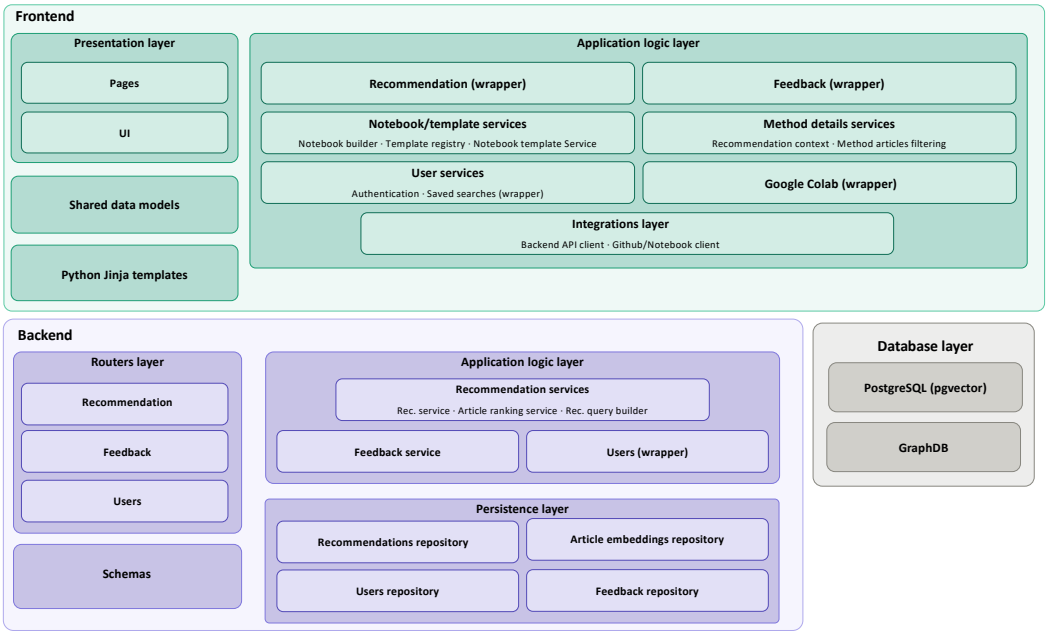


**Figure 4.1.3:** Package Diagram of MLguide.

## 4.2 Knowledge Graph and Database

This section describes the two data stores used in MLguide. The first is the ML knowledge graph stored in GraphDB, which is based on ML Ontology and extended for use in MLguide. The second is the PostgreSQL database, which stores article abstracts and vector embeddings used for semantic retrieval.

### 4.2.1 ML Ontology Extension

ML Ontology was selected as the knowledge base for MLguide because it provides the broadest coverage of ML concepts among the ontologies reviewed in Section 2.3.5, and because its structure directly supports the kind of structured method retrieval that MLguide requires. It is open source and published under an MIT licence (**humm2026industry_ready_machine_learning_ontology**), which allows for use and modification as needed. The limitations identified in Section 2.3.5, including the absence of reinforcement learning instances and incomplete coverage of `ML_approach`, required extension before the ontology could be used in MLguide. Some extensions were also made to support better knowledge structure and specific MLguide functionality. They are described in more detail in this section.

Three new classes were added under a separate namespace (`mla:`), short for *machine learning articles*, to make clear that they are not part of the original ML Ontology. Table 4.2.1 describes these classes.

**Table 4.2.1:** New classes added under the `mla:` namespace.

| Class | Description |
|---|---|
| `mla:Article` | Represents a research article from the literature dataset. Linked to one or more `mla:Method` instances that are mentioned in the article. |
| `mla:Method` | Represents an ML method as extracted from article abstracts. Linked to a corresponding `ML_approach` instance via `skos:exactMatch`. |
| `mla:ML_task_family` | Groups related `ML_task` instances under a common family label, helps selecting notebook templates during notebook generation. |

`mla:Method` and `ML_approach` represent the same concept but serve different roles. The reason for keeping them separate is practical: `ML_approach` instances carry relations to other ML concepts in the ontology, such as which tasks a method can be used for and which learning area it belongs to. Adding article relations directly to these instances would mix two different kinds of knowledge on the same node, making the ontology harder to navigate during development. Instead, article relations are kept on `mla:Method`, and the two classes are connected via `skos:exactMatch`. If needed, this separation could be removed by transferring the article relations directly to the corresponding `ML_approach` instances.

Figure 4.2.1 shows the class structure of the extended ontology. The upper part shows the classes from the original ML Ontology that are directly connected to the three new `mla:` classes: `ML_approach`, `ML_task`, and `ML_area`. The lower part shows the new `mla:` classes and how they connect to these. `mla:Method` is linked to `ML_approach` via `skos:exactMatch`, and to `mla:Article` via `mentionsMethod`. `mla:ML_task_family` is linked to `ML_task` via `has_family`, and `mla:Article` is linked to `ML_area` via `has_ml_area`.

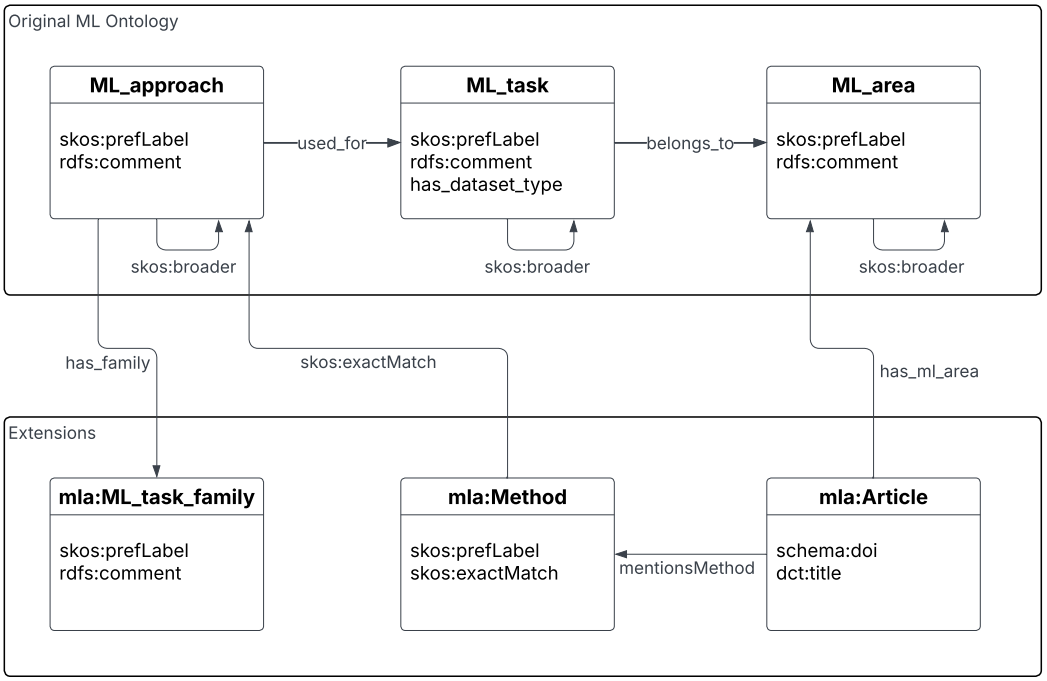


**Figure 4.2.1:** UML Class Diagram of the ML Ontology, including added classes and properties for MLguide.

Figure 4.2.2 shows a concrete example of how the extended ontology represents an article and its relations. The `mla:Article` instance represents a research article identified by its DOI and title. It is linked to `supervised_learning` via `has_ml_area`, and to `method_linear_regression`, an `mla:Method` instance, via `mentionsMethod`. The method instance is in turn linked to the `ML_approach` instance `linear_regression` via `skos:exactMatch`, which carries the ontology relations: `used_for` links it to the `ML_task regression`, which belongs to `supervised_learning` via `belongs_to`, and `has_family` links it to the `mla:ML_task_family` instance `regression_family`.

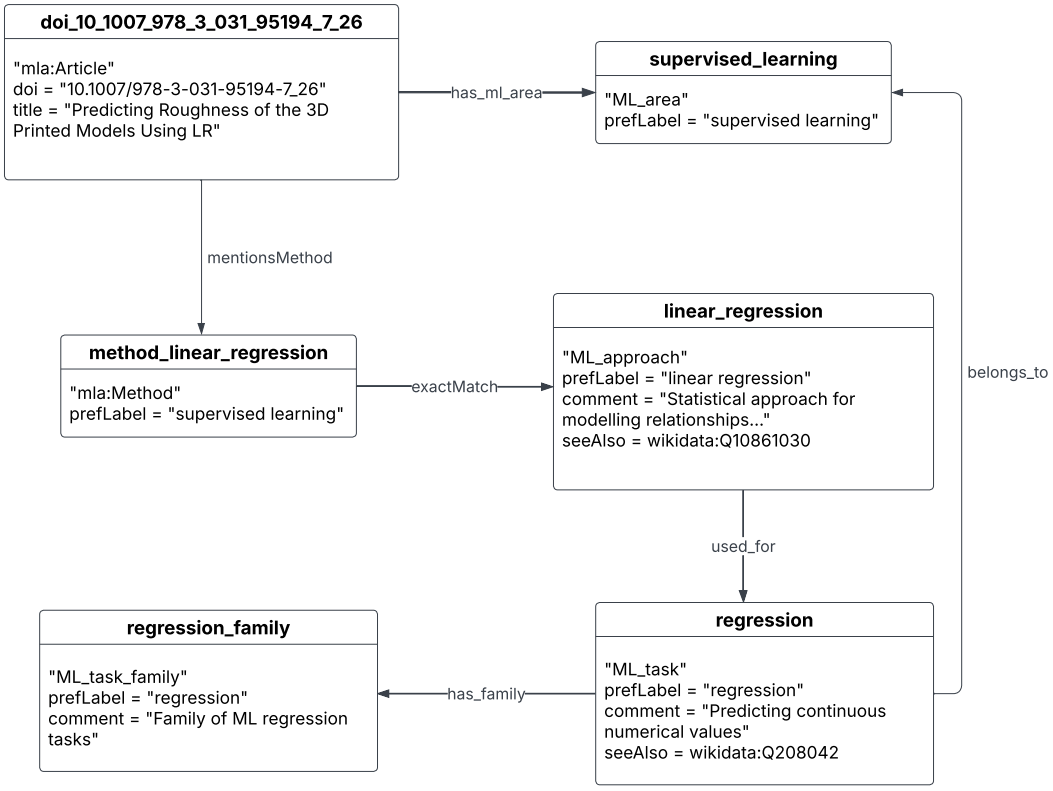


**Figure 4.2.2:** Example of an Article instance in the ML Ontology.

New `ML_approach` instances were added to address the coverage gaps identified in Section 2.3.5. A first set was derived from the method extraction results of the pre-master project (**bergsto2026mlselection**), where ML methods were identified from article abstracts in the literature dataset. Methods that appeared in the dataset but had no corresponding `ML_approach` instance in ML Ontology were added. Reinforcement learning methods and tasks were added based on established taxonomies of RL algorithms (**zhang2020taxonomy_reinforcement_learning_algorithm** **geron2019hands_on_machine_learning**), following the same classification used in Section 2.1.2. In total, 65 new instances were added, grouped by theme in Table 4.2.2.

**Table 4.2.2:** New `ML_approach` instances added to ML Ontology, grouped by theme.

| Theme | New instances | n |
|---|---|---|
| Reinforcement | deep reinforcement learning, Q-learning, REINFORCE, deep Q-network, policy gradient, proximal policy optimization, soft actor-critic, actor-critic, deep deterministic policy gradient, twin delayed DDPG, A3C | 11 |
| Deep learning | deep neural network, attention mechanism, graph attention network, graph neural network, graph convolutional network, GraphSAGE, U-Net, variational autoencoder, normalizing flow, diffusion model, DeepAR | 11 |
| Evolutionary | genetic algorithm, particle swarm optimization, simulated annealing, differential evolution, ant colony optimization, firefly algorithm, harmony search, evolutionary strategy, CMA-ES | 9 |
| Anomaly detection | isolation forest, local outlier factor, one-class SVM, HBOS, Hotelling $T^2$, squared prediction error, minimum covariance determinant, elliptic envelope | 8 |
| Time series | Kalman filter, temporal convolutional network, N-BEATS, seasonal ARIMA, state space model, VAR, Holt-Winters | 7 |
| Explainability | explainable AI, SHAP, Granger causality, DoWhy, CausalImpact, causal forest | 6 |
| Probabilistic | Bayesian method, Gaussian process regression, partial least squares, elastic net, robust regression | 5 |
| Clustering | k-medoids, Gaussian mixture model clustering, t-SNE, UMAP | 4 |
| Ensemble | gradient boosting, extra trees, bagging | 3 |
| Other | naive Bayes | 1 |
| **Total** | | **65** |

For methods that form hierarchical families, `skos:broader` relations were added to preserve the inheritance structure of the ontology. For example, `deep_q_network` was linked as broader to `q_learning`, and `proximal_policy_optimization` to `policy_gradient`. After adding new instances, the inheritance rule described in Section 2.3.5 was applied to propagate `used_for` and `belongs_to` relations to the new instances through their `skos:broader` hierarchies.

Where available, links to corresponding Wikidata entities were added to new in-

stances via `rdfs:seeAlso`, following the same approach used in the original ontology (**humm2026industry_ready_machine_learning_ontology**). Candidates were retrieved using the Wikidata search API and scored by string similarity between labels and whether the Wikidata description contained predefined ML-related terms. Results were reviewed manually before being added.

Table 4.2.3 shows the resulting coverage across all 187 `ML_approach` instances after the extension and applying the inference rule. For reference, the original coverage across 122 instances is shown in Table 2.3.2. Coverage remains incomplete for several properties. Populating properties requires a lot of manual work for each instance, both for existing instances that lacked them and for all newly added ones. Effort on this was therefore prioritised towards reinforcement learning, which had no instances at all in the original ontology, and less towards filling gaps in the remaining properties.

**Table 4.2.3:** Coverage of `ML_approach` properties across 187 instances after ontology extension and inference rule application.

| Property | Coverage |
|---|---|
| `skos:prefLabel` | 187/187 (100%) |
| `used_for` | 131/187 (70%) |
| `rdfs:comment` | 130/187 (70%) |
| `rdfs:seeAlso` | 90/187 (48%) |
| `skos:altLabel` | 66/187 (35%) |
| `skos:broader` | 57/187 (30%) |
| `performance` | 50/187 (27%) |
| `possible_if` | 28/187 (15%) |
| `not_recommended_if` | 5/187 (3%) |

`mla:ML_task_family` instances were also added to group related `ML_task` instances under a common label. This was motivated by the large amount of similar tasks, especially for classification and regression. Grouping them into families provides a clearer structure. Task families also support notebook generation by providing a better structure for selecting the correct template for a given method and task combination, since many methods can be applied to more than one type of task. This is described in more detail in Section 4.4.

Families were defined by inspecting the `skos:broader` hierarchy to identify which tasks were already grouped under common parent tasks, and also by grouping tasks of similar character where no such hierarchy existed. Each family was given a name that reflects the shared nature of its tasks. Table 4.2.4 lists the task families and their assigned tasks.

**Table 4.2.4:** ML_task_family instances and their assigned ML_task instances.

| ML_task_family | Assigned ML_task instances | n |
|---|---|---|
| classification_family | classification, binary_classification, multi-class_classification, tabular_classification, text_classification, image_classification, audio_classification, time_series_classification, video_classification, vertex_classification | 10 |
| regression_family | regression, tabular_regression, text_regression, image_regression, audio_regression, video_regression | 6 |
| clustering_ unsupervised_ discovery_family | clustering, tabular_clustering, association_rule_learning, community_detection, topic_modeling | 5 |
| detection_ localization_family | object_detection, anomaly_detection, action_recognition | 3 |
| feature_engineering_ family | feature_extraction, feature_reduction, feature_selection | 3 |
| structured_ prediction_family | image_segmentation, named_entity_recognition, translation | 3 |
| graph_tasks_family | graph_matching, link_prediction, vertex_classification | 3 |
| nlp_analysis_family | sentiment_analysis, text_analytics | 2 |
| ranking_ recommendation_ similarity_family | ranking, recommendation, sentence_similarity | 3 |
| time_series_family | time_series_analysis, time_series_forecasting, time_series_classification | 3 |
| tracking_family | video_object_tracking | 1 |
| **Total** | Note: some tasks appear in multiple families | **42** |

### 4.2.2 Database Structure

Article data is stored across several tables in PostgreSQL, shown in Figure 4.2.3. The `article_abstracts` table stores the raw abstract text for each article, used during reranking. The `abstract_embeddings` and `title_embeddings` tables store 768-dimensional vector embeddings of the abstract and title respectively, used for semantic similarity search via pgvector (**pgvector**). The `users` table stores a minimal user record created on first login. The `saved_searches` table stores previously saved searches with problem descriptions, allowing users to reload them

without re-entering their inputs. The `method_feedback` table stores user feedback on recommended methods, capturing all context related to the user's problem description and the specific method.

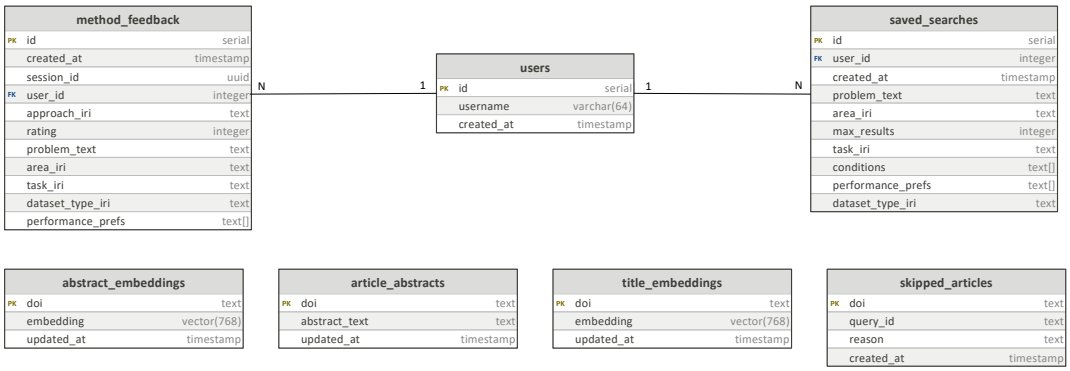


**Figure 4.2.3:** Entity-Relationship Diagram of the PostgreSQL database used in MLguide.

## 4.3 Recommendation Process

The recommendation process takes the user's input and produces a ranked list of ML methods. The process has two main stages. In the first stage, candidate methods are retrieved from the ontology based on the parameters the user has selected. In the second stage, if a problem description is provided, the candidates are reranked using semantic similarity to relevant research articles. A feedback boost is applied as a final adjustment to the ranking.

Figure 4.3.1 shows the sequence of steps in the recommendation process, covering ontology-based candidate retrieval and semantic similarity ranking. Feedback is not included in detail in the diagram. It is described separately in Section 4.3.4.

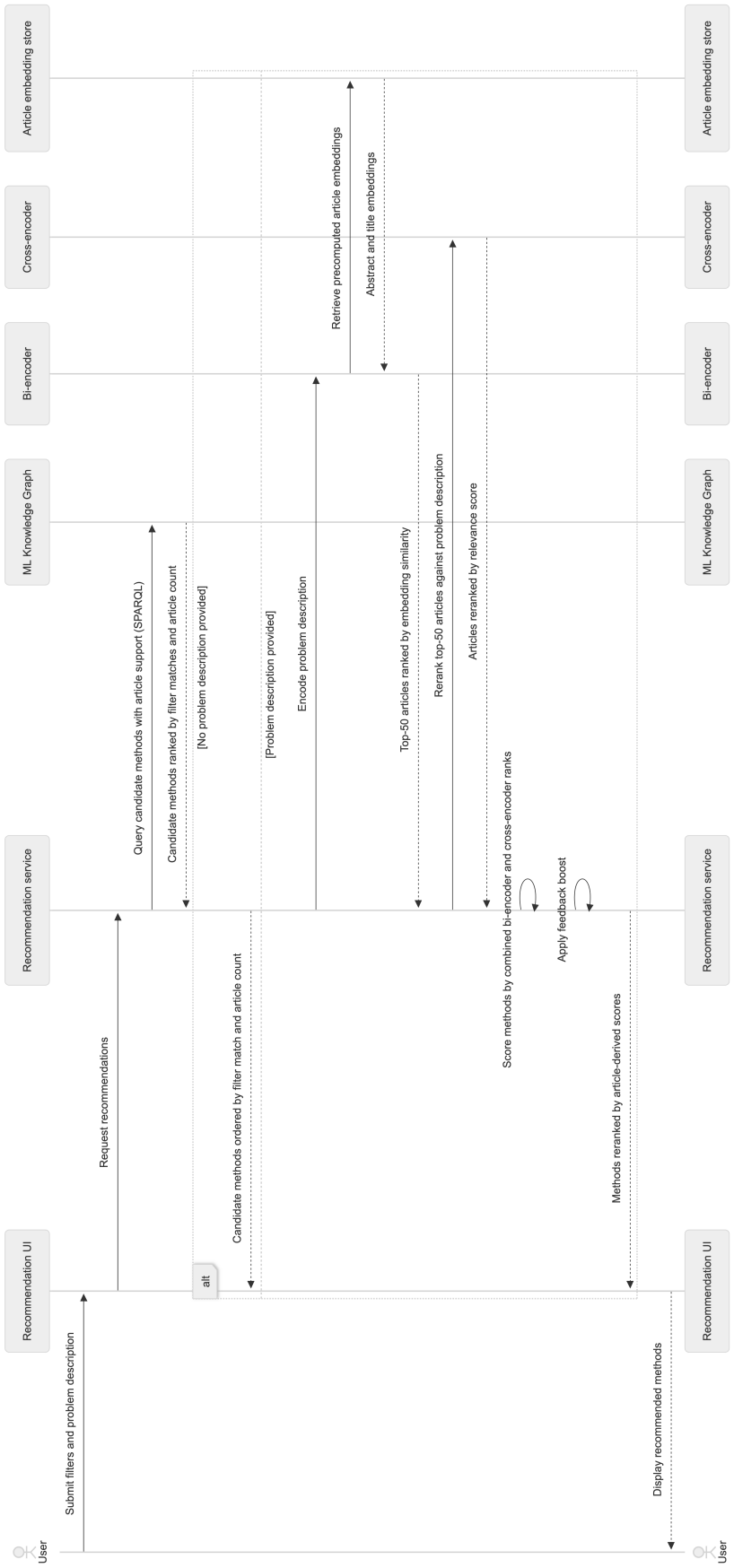


**Figure 4.3.1:** Sequence diagram of the recommendation process in MLguide, showing ontology-based candidate retrieval and semantic similarity ranking.

### 4.3.1 Ontology-Based Candidate Retrieval

The first step is to retrieve candidate methods from the knowledge graph using a SPARQL query against GraphDB. The query goes through the graph from articles to methods to approaches. It finds articles tagged with the selected ML area, follows the `mentionsMethod` relation to `mla:Method` instances, and then follows `skos:exactMatch` to the corresponding `ML_approach` instances. A candidate approach is only included if at least one article in the knowledge graph mentions a method that maps to it, which grounds the candidate set in the research literature.

The properties used to filter and rank candidates are listed in Table 4.3.1. They were selected because they describe characteristics of an ML method that can be matched against the user's problem context. ML area is the only hard filter, since all articles in the knowledge graph are tagged with an ML area, restricting to the selected area does not risk excluding relevant candidates. The remaining properties have incomplete coverage, as shown in Table 4.2.3, and are therefore used as soft signals that boost the ranking of matching methods without excluding non-matching ones.

**Referere til 2.1.2 ? Og den manglende delen om attributter på metoder**

**Table 4.3.1:** Ontology properties used for filtering and ranking in candidate retrieval.

| Property | Source | Example values |
|---|---|---|
| `has_ml_area` | `mla:Article` | `supervised_learning`, `reinforcement_learning` |
| `used_for` | `ML_approach` | `classification`, `regression`, `anomaly_detection` |
| `performance` | `ML_approach` | `fast_training`, `accurate`, `explainable` |
| `has_dataset_type` | `ML_task` via `used_for` | `tabular`, `image`, `text` |

Task, performance, and dataset type filters are applied as optional matches. For each candidate method, the query counts how many of the user's selected values match the corresponding ontology properties. These counts are used to sort candidates, so methods that match more of the user's filters rank higher. Dataset type is accessible indirectly through the tasks linked via `used_for`, as shown in the table.

The other properties of `ML_approach`: `possible_if` and `not_recommended_if` are retrieved and returned alongside each method as informational notes, but are

not used for filtering or ranking. This due to the low coverage of these properties shown in Table 4.2.3. Therefore using them for filtering at this would risk excluding valid candidates.

A filter is also applied to remove overly general methods. This means that if a specific neural network variant is already present in the candidate set from the same article, the generic `neural_network` method is removed not counted to reduce noise. In the end, candidates are sorted by how well they match the user's filters, with the number of supporting articles as a tiebreaker. If no problem description is provided, this ordered list is returned directly, only adjusted for feedback as described in Section 4.3.4.

### 4.3.2 Article Similarity Scoring

When a problem description is provided, articles in the database are ranked by their semantic similarity to the query, using the two-stage retrieval pipeline combining a bi-encoder and a cross-encoder, as described in Section 2.4.4. Similarity is measured using cosine similarity, which is described as the most common metric in Section 2.4.1. It compares the angle between two vectors regardless of their magnitude (**jurafsky_martin2026speech_language_processing**).

In the first stage, the problem description is encoded using `all-mpnet-base-v2` (**sbert_mpnet**), a sentence-transformer model based on MPNet (**song2020mpnet**). This model was used in the pre-master project (**bergsto2026mlselection**) to compute the article embeddings stored in the database, so the same model must be used here to ensure the problem description is encoded in the same embedding space. The model was also selected for its quality. It produces 768-dimensional embeddings and has been shown to perform well on abstract-level semantic similarity tasks in scientific literature (**galli2024performance_pre_trained_sentence_transformer_mod** **colangelo2025comparative_analysis_sentence_transformer_models**). Article scores are computed as a weighted sum of cosine similarities between the query embedding and the precomputed abstract and title embeddings:

$$\text{score}_a = w_{\text{abs}} \cdot \text{sim}_{\text{abs}}(a) + w_{\text{title}} \cdot \text{sim}_{\text{title}}(a) \tag{4.1}$$

where $w_{\text{abs}} = 0.7$ and $w_{\text{title}} = 0.3$. The higher weight on abstracts reflects that abstracts contain more information about the method and application context than titles. These weights are set as defaults, but can be adjusted. The top $k = 50$ articles are retained for the reranking step.

In the second stage, the top-$k$ articles are reranked using `ms-marco-MiniLM-L6-v2`

(**sbert_ce_msmarco**), a cross-encoder fine-tuned on the MS MARCO passage ranking dataset (**bajaj2018ms_marco_machine_reading_comprehension_dataset**). The model was chosen because it achieves strong ranking performance while being lightweight enough for query-time inference over a small candidate set. Among 14 publicly available cross-encoder models evaluated on MS MARCO, it was found to offer the best performance (**thakur2021beir_heterogenous_benchmark_zero_shot_eva**). As described in Section 2.4.4, a cross-encoder processes the query and each article jointly, producing more accurate relevance scores than the bi-encoder alone.

### 4.3.3 Method Scoring

After article reranking, each candidate method is scored based on the articles that mention it. Combining rank positions from both stages gives weight to both broad semantic similarity from the bi-encoder and precise relevance scoring from the cross-encoder. Using rank positions rather than the raw similarity and relevance scores from each model also avoids differences in scale between the two sources, consistent with the principle behind Reciprocal Rank Fusion described in Section 2.5:

$$\text{score}_m = \sum_{\substack{a \in \text{top-}k \\ m \in a}} \left( \frac{w_b}{r_b(a) + 1} + \frac{w_c}{r_c(a) + 1} \right) \tag{4.2}$$

where $r_b(a)$ and $r_c(a)$ are the zero-based rank positions of article $a$ in the bi-encoder ranking and cross-encoder ranking respectively. The weights $w_b$ and $w_c$ are normalised with $w_b + w_c = 1$, and both default to 0.5. The choice to combine both signals was made by manually checking output for queries of different types and levels of detail, since no ground-truth ranking data exists. Using only the cross-encoder score directly was the initial approach and is a possibility. Both the signal weights and this choice are configurable. The score calculation ensures that articles ranked higher contribute more to the method score, and the contribution decreases gradually for lower-ranked articles.

### 4.3.4 Feedback Boost

After methods are ranked by similarity score, user feedback can adjust the final ordering. When a user rates a method on a 1–5 scale, the rating is stored and used in subsequent recommendations. As discussed in Section 2.5, rating estimates based on few observations are unreliable. A smoothed score is therefore computed using Bayesian smoothing towards a neutral prior (**murphy2012machine_learning_probabilistic**

$$\hat{r}_m = \frac{s \cdot \mu_0 + n \cdot \bar{r}_m}{s + n} \tag{4.3}$$

where $\bar{r}_m$ is the observed mean rating, $n$ is the number of ratings, $s = 5$ is the prior strength, and $\mu_0 = 3$ is the prior mean on a 1–5 scale. When $n$ is small, the estimate is pulled towards $\mu_0$. As $n$ grows, the observed mean dominates. The smoothed rating is normalised to $[-1, 1]$ relative to the neutral rating of 3. The prior strength of 5 means that approximately five ratings are needed before feedback has a meaningful effect on the score. This is a low threshold, set with the expectation that the system will initially have a small user base. The value is a default and can be adjusted.

The feedback score is combined with the similarity-based ranking using a rank-based score, so that the two signals are on a comparable scale:

$$\text{combined\_score}(m) = \frac{1}{\text{rank}(m) + 1} + w_f \cdot \text{feedback\_score}(m) \tag{4.4}$$

where $\text{rank}(m)$ is the zero-based position after similarity ranking, and $w_f = 0.15$ is an adjustable weight value. The low weight means that feedback can adjust a method's position by a few places, but cannot override the similarity-based ranking entirely.

Figure 4.3.2 illustrates the full ranking process.

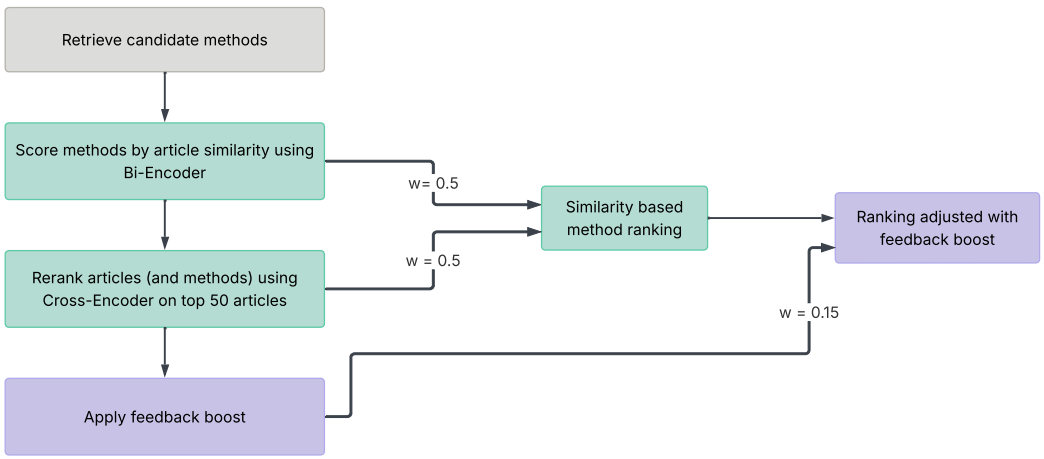


**Figure 4.3.2:** Flowchart of the ranking process in MLguide, including similarity scoring and feedback boost.

## 4.4 Notebook Generation

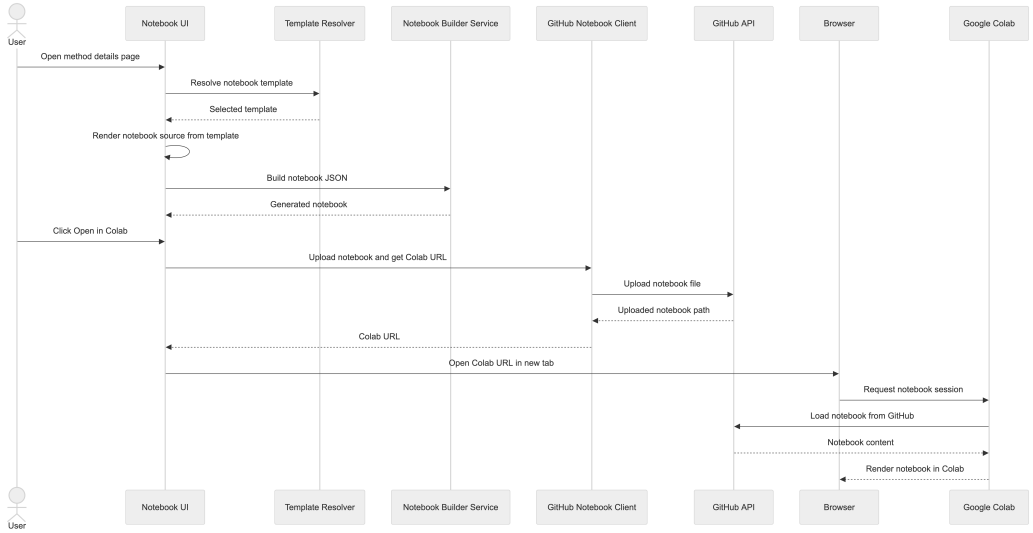


**Figure 4.4.1:** Sequence diagram of the notebook generation process in MLguide.

## 4.5 Article Retrieval and Knowledge Extraction

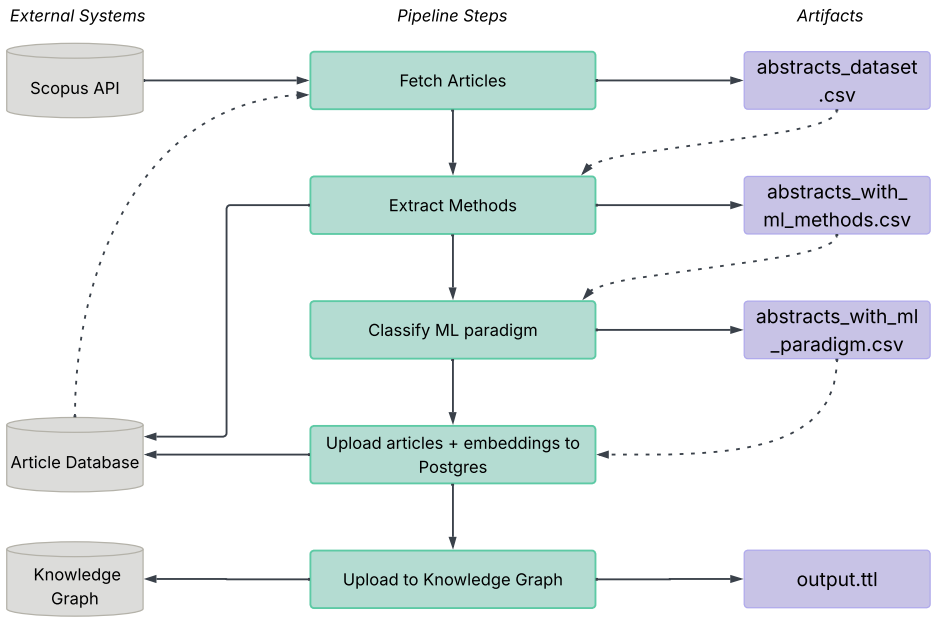


**Figure 4.5.1:** Flowchart of the article retrieval and knowledge extraction process in MLguide.

## 4.6 Web Interface

CHAPTER

# FIVE

# RESULTS

## 5.1 Project 1 (P1): Unsupervised Learning and Anomaly Detection for Time Series Data

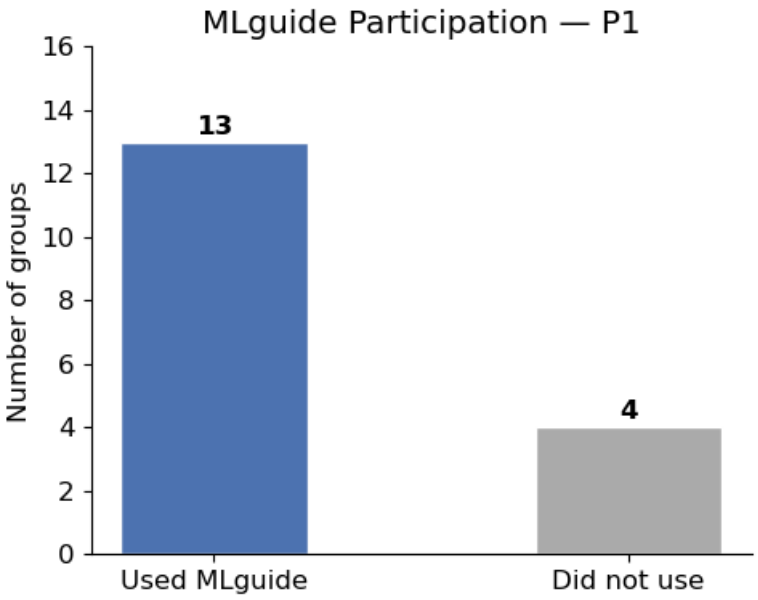


**Figure 5.1.1:** Number of groups participating.

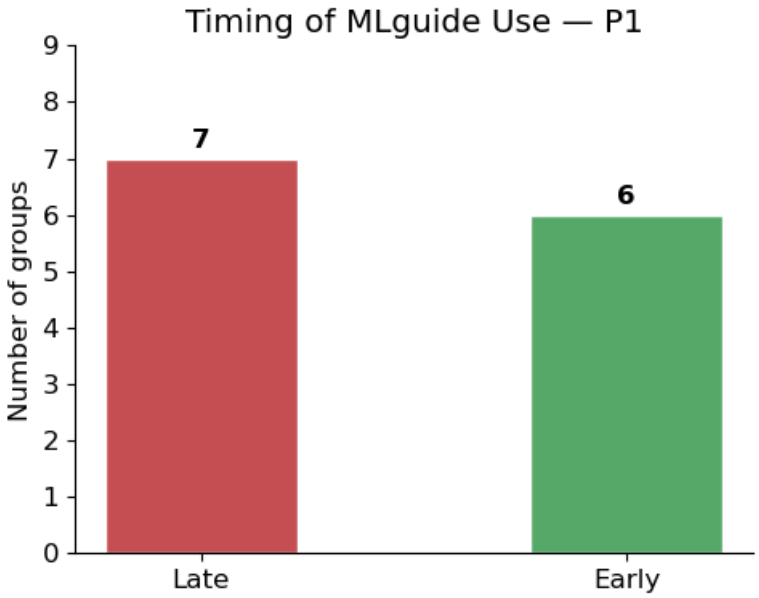


**Figure 5.1.2:** When in the project groups used MLguide, before or after choosing methods.

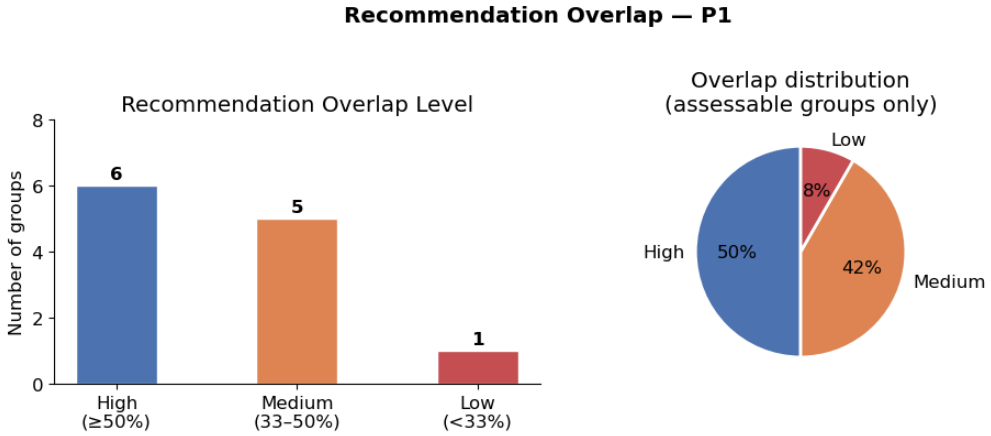


**Figure 5.1.3:** Overlap of recommended methods with the methods chosen by the groups.

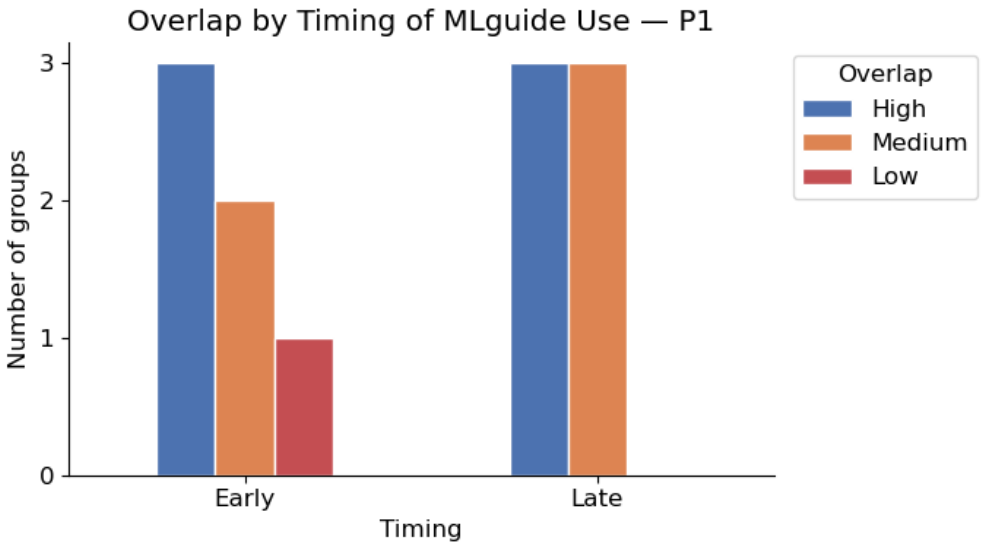


**Figure 5.1.4:** Timing and overlap of recommendations with chosen methods combined.

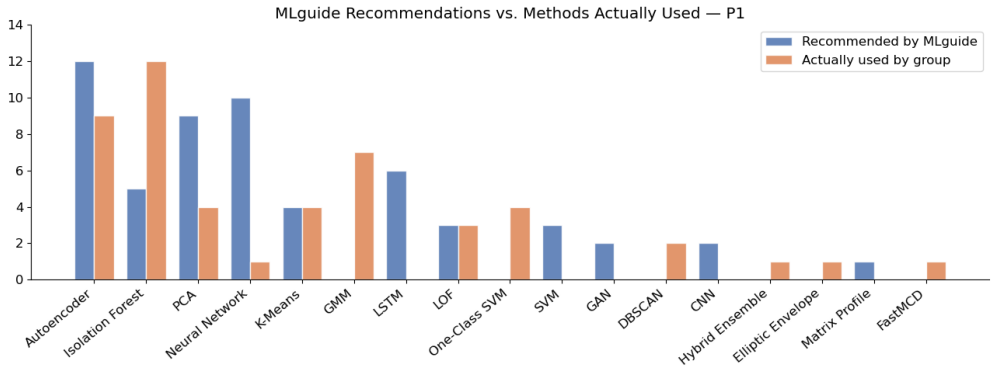


**Figure 5.1.5:** Recommended vs. Used Methods.

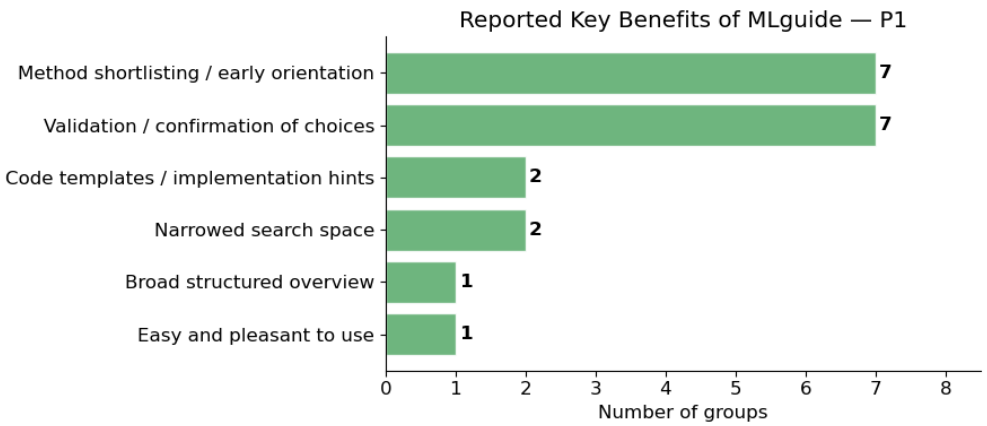


**Figure 5.1.6:** Reported benefits of using MLguide.

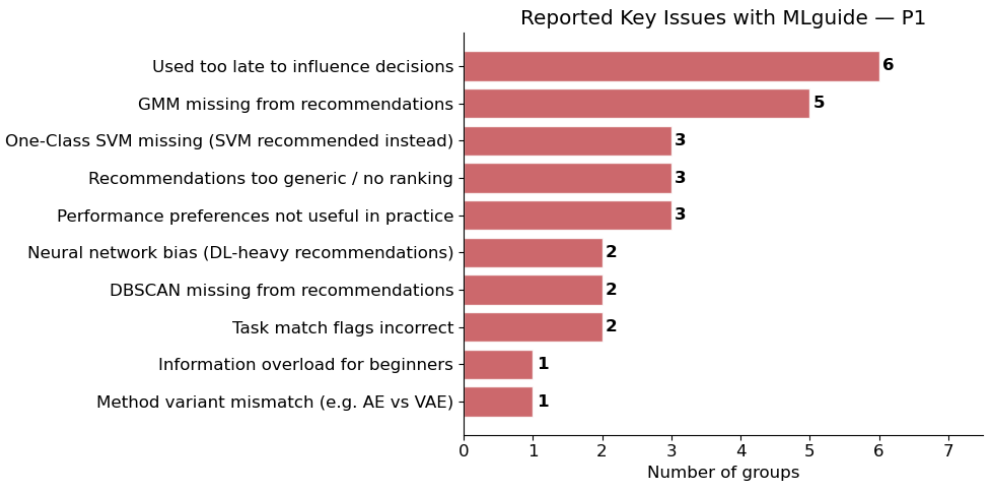


**Figure 5.1.7:** Reported issues of using MLguide.

## 5.2 Project 2 (P2): Reinforcement Learning Across Product Lifecycle

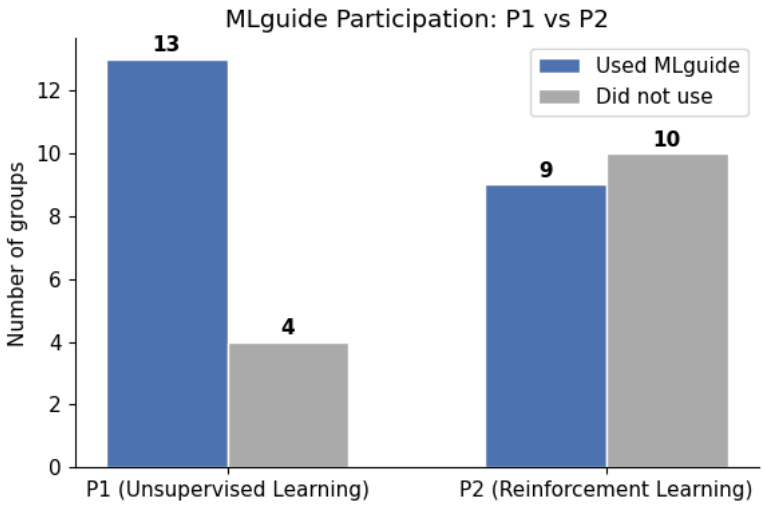


**Figure 5.2.1:** Number of groups participating.

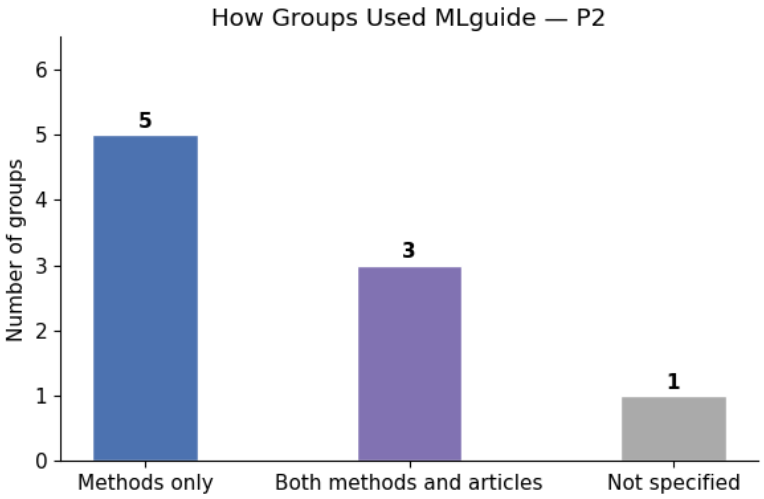


**Figure 5.2.2:** How groups used MLguide.

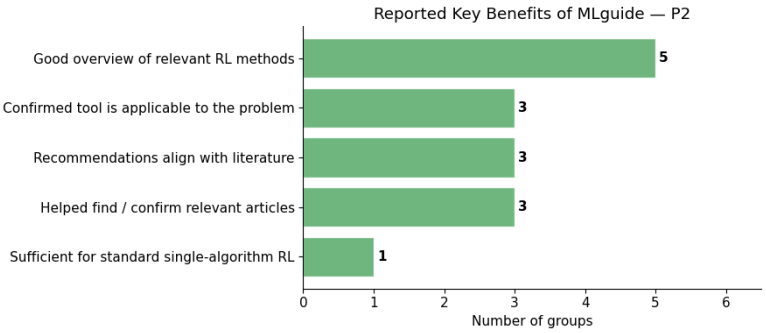


**Figure 5.2.3:** Key benefits reported by groups.

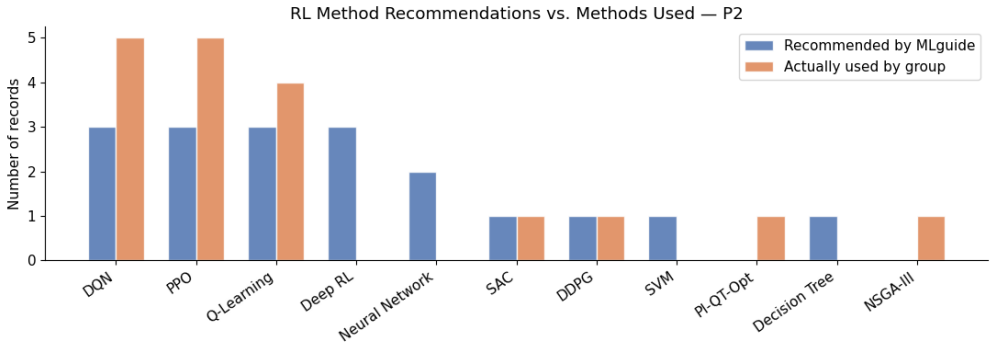


**Figure 5.2.4:** Methods Used vs. Recommended.

CHAPTER

# SIX

# DISCUSSION

### 6.0.1 Feedback and Limitations

**Skrives om**

The feedback from both projects gave useful insight into the usability of MLguide and what could be improved. However, two limitations applied.

The first concerned the feedback format. Feedback was collected as part of the students' project reports, which fit naturally within the course structure and allowed students to reflect on MLguide in the context of their own problem and data. Since each group worked on a different problem, an open format seemed more suitable than a fixed set of questions. The trade-off was that responses varied in detail and focus, making it more difficult to draw systematic conclusions.

The second limitation was that the feedback arrived quite late in the development period. This was probably unavoidable, as a working system had to be in place before it could be tested. The feedback led to some adjustments, but larger changes such as new features or structural redesigns were not feasible within the remaining time. These can instead be left as directions for future work.

CHAPTER

# SEVEN

# CONCLUSIONS

# APPENDICES